```
1  package com.aicompanion
2
3  import android.content.Context
4  import com.aicompanion.action.AIActionManager
5  import com.aicompanion.affection.AffectionManager
6  import com.aicompanion.gamify.AchievementManager
7  import com.aicompanion.memory.ContextManager
8  import com.aicompanion.memory.MemorableMomentsManager
9  import com.aicompanion.network.ApiClient
10 import com.aicompanion.plugin.*
11 import com.aicompanion.rag.PersonaRagManager
12 import com.aicompanion.settings.SettingsManager
13 import com.aicompanion.sticker.StickerManager
14 import com.aicompanion.ui.FavoriteManager
15 import com.aicompanion.ui.NicknameManager
16 import com.aicompanion.voice.VoiceManager
17
18 object AppContainer {
19     var appContext: Context? = null
20         private set
21     var settingsManager: SettingsManager? = null
22         private set
23     private var _apiClient: ApiClient? = null
24     private var _affectionManager: AffectionManager? = null
25     private var _achievementManager: AchievementManager? = null
26     private var _contextManager: ContextManager? = null
27     private var _personaRagManager: PersonaRagManager? = null
28     private var _favoriteManager: FavoriteManager? = null
29     private var _nicknameManager: NicknameManager? = null
30     private var _voiceManager: VoiceManager? = null
31     private var _momentsManager: MemorableMomentsManager? = null
32     private var _actionManager: AIActionManager? = null
33     private var _stickerManager: StickerManager? = null
34     private var _searchMemoryPlugin: SearchMemoryPlugin? = null
35     private var _searchDiaryPlugin: SearchDiaryPlugin? = null
36     private var _nicknamePlugin: NicknamePlugin? = null
37     private var _generateImagePlugin: GenerateImagePlugin? = null
38
39     private var _cachedPersonaId: String? = null
40
41     val apiClient: ApiClient? get() = _apiClient
42     val affectionManager: AffectionManager
43         get() = getOrCreatePersonaAware(::_affectionManager, { _affectionManager = it }) { ctx, pid ->
44             AffectionManager(ctx, pid)
45         }
46     val achievementManager: AchievementManager
47         get() = getOrCreatePersonaAware(::_achievementManager, { _achievementManager = it }) { ctx, pid ->
48             AchievementManager(ctx, pid)
49         }
50     val stickerManager: StickerManager
51         get() = _stickerManager ?: StickerManager(requireContext()).also { _stickerManager = it }
52     val contextManager: ContextManager
53         get() = _contextManager ?: ContextManager(requireContext()).also { _contextManager = it }
54     val favoriteManager: FavoriteManager
55         get() = getOrCreatePersonaAware(::_favoriteManager, { _favoriteManager = it }) { ctx, pid ->
56             FavoriteManager(ctx, pid)
57         }
58     val nicknameManager: NicknameManager
59         get() = getOrCreatePersonaAware(::_nicknameManager, { _nicknameManager = it }) { ctx, pid ->
60             NicknameManager(ctx, pid)
61         }
62     val voiceManager: VoiceManager
```

```
63          get() = _voiceManager ?: VoiceManager(requireContext()).also { _voiceManager = it }
64      val momentsManager: MemorableMomentsManager
65          get() = getOrCreatePersonaAware(::_momentsManager, { _momentsManager = it }) { ctx, pid ->
66              MemorableMomentsManager(ctx, pid)
67          }
68      val actionManager: AIActionManager
69          get() = _actionManager ?: AIActionManager(requireContext()).also { _actionManager = it }
70      val personaRagManager: PersonaRagManager
71          get() = getOrCreatePersonaAware(::_personaRagManager, { _personaRagManager = it }) { ctx, pid ->
72              PersonaRagManager(ctx, pid)
73          }
74
75      private fun requireContext(): Context {
76          return appContext ?: throw IllegalStateException("AppContainer not initialized. Call initialize() first.")
77      }
78
79      private fun readActivePersonaId(): String {
80          return appContext?.getSharedPreferences("app_prefs", android.content.Context.MODE_PRIVATE)
81              ?.getString("active_persona_id", "default") ?: "default"
82      }
83
84      private inline fun <T : Any> getOrCreatePersonaAware(
85          currentGetter: () -> T?,
86          setter: (T) -> Unit,
87          factory: (Context, String) -> T
88      ): T {
89          val ctx = requireContext()
90          val currentPid = readActivePersonaId()
91          val current = currentGetter()
92          if (current != null && _cachedPersonaId == currentPid) return current
93          val newInstance = factory(ctx, currentPid)
94          setter(newInstance)
95          _cachedPersonaId = currentPid
96          return newInstance
97      }
98
99      fun onPersonaChanged() {
100         _affectionManager = null
101         _achievementManager = null
102         _favoriteManager = null
103         _nicknameManager = null
104         _momentsManager = null
105         _personaRagManager = null
106         _cachedPersonaId = null
107     }
108
109     fun initialize(appContext: Context) {
110         this.appContext = appContext
111         settingsManager = SettingsManager(appContext)
112         com.aicompanion.network.ProviderAdapter.init(appContext)
113         // 初始化应用分类器（用户自定义 App 分类的持久化存储）
114         com.aicompanion.screen.AppCategoryClassifier.init(appContext)
115         // 把「工具调用轮次」设置同步给 ApiClient 的静态默认值（其它调用点无需再传参）
116         try {
117             val sm = settingsManager
118             if (sm != null) {
119                 com.aicompanion.network.ApiClient.defaultToolLoopMaxIterations = sm.aiToolLoopMaxRounds
120                 // 把 UI 树采集参数应用到无障碍服务（服务未连接时会在 onServiceConnected 再应用一次）
121                 com.aicompanion.screen.ScreenRecognitionService.applyConfig(sm)
122             }
123         } catch (_: Exception) {}
124         // 迁移旧版 App UI 知识到统一技能库（只执行一次）
```

```
        try {
            com.aicompanion.skill.SkillStore(appContext).migrateFromAppSkillsIfNeeded()
        } catch (_: Exception) {}
        // RagConfig已在CompanionApp.onCreate中初始化,这里只做模型降级检测
        checkOnnxModelDegradation(appContext)
        registerBuiltinPlugins(appContext)
        // 初始化 API 客户端（确保 HomeActivity 等页面可直接使用）
        rebuildApiClient()
    }

    /**
     * ONNX模型自动降级: 如果选择了local模式但模型未就绪,降级到tfidf
     */
    private fun checkOnnxModelDegradation(context: Context) {
        if (com.aicompanion.rag.RagConfig.embeddingMode == "local") {
            try {
                val onnxEmbedder = com.aicompanion.rag.OnnxEmbedder(context)
                if (!onnxEmbedder.isModelReady()) {
                    com.aicompanion.util.AppLogger.w("AppContainer", "ONNX模型未就绪,自动降级到tfidf模式")
                    com.aicompanion.rag.RagConfig.embeddingMode = "tfidf"
                } else {
                    com.aicompanion.util.AppLogger.i("AppContainer", "ONNX模型已就绪,使用local模式")
                }
            } catch (e: Exception) {
                com.aicompanion.util.AppLogger.e("AppContainer", "ONNX模型检测失败,降级到tfidf: ${e.message}")
                com.aicompanion.rag.RagConfig.embeddingMode = "tfidf"
            }
        }
    }

    fun rebuildApiClient() {
        val sm = settingsManager ?: return
        if (sm.chatApiUrl.isNotBlank()) {
            _apiClient = ApiClient(sm.chatApiUrl, sm.chatApiKey, sm.chatModel,
                sm.llmTemperature, sm.llmTopP, sm.llmFrequencyPenalty, sm.llmPresencePenalty, sm.llmMaxTokens,
                sm.apiProvider)
        }
    }

    private fun registerBuiltinPlugins(context: Context) {
        PluginRegistry.clear()
        PluginRegistry.register(AlarmPlugin(context))
        PluginRegistry.register(AlarmAtTimePlugin(context))
        PluginRegistry.register(SchedulePlugin(context))
        PluginRegistry.register(WebSearchPlugin(context))
        _searchMemoryPlugin = SearchMemoryPlugin()
        PluginRegistry.register(_searchMemoryPlugin!!)
        _searchDiaryPlugin = SearchDiaryPlugin()
        PluginRegistry.register(_searchDiaryPlugin!!)
        PluginRegistry.register(CurrentTimePlugin())
        _nicknamePlugin = NicknamePlugin()
        PluginRegistry.register(_nicknamePlugin!!)
        val sendStickerPlugin = SendStickerPlugin(context)
        PluginRegistry.register(sendStickerPlugin)
        _generateImagePlugin = GenerateImagePlugin(context)
        PluginRegistry.register(_generateImagePlugin!!)
        // 记忆工具插件：AI 自动添加日历事件/里程碑/难忘时刻/时光胶囊
        PluginRegistry.register(CalendarEventPlugin(context))
        PluginRegistry.register(MilestonePlugin(context))
        PluginRegistry.register(MemorableMomentPlugin(context))
        PluginRegistry.register(TimeCapsulePlugin(context))
        PluginRegistry.register(AlbumPlugin(context))
```

```
187             // 手机操作工具：LLM 通过工具调用形式操作手机
188             PluginRegistry.register(PhoneOperationPlugin(context))
189             // 技能库：跨 App 自动化技能的查询/创建/修改
190             PluginRegistry.register(SkillPlugin(context))
191             // 命令执行：在设备上跑 shell（默认关闭，需在设置中开启）
192             PluginRegistry.register(ShellPlugin(context))
193             // 文件读写：agent 级文件操作（工作区沙箱内，不经过 shell）
194             PluginRegistry.register(FilePlugin(context))
195             // 系统动作：拨号/开应用/系统设置等确定性的系统能力，走官方 Intent 而非模拟点击
196             PluginRegistry.register(SystemActionPlugin(context))
197         }
198
199         fun setSearchMemoryCallback(callback: suspend (String, Int) -> String) {
200             _searchMemoryPlugin?.onSearchMemory = callback
201         }
202
203         fun setSearchDiaryCallback(callback: suspend (String, Int) -> String) {
204             _searchDiaryPlugin?.onSearchDiary = callback
205         }
206
207         fun setNicknameCallback(callback: (List<String>) -> Unit) {
208             _nicknamePlugin?.onNicknamesGenerated = callback
209         }
210
211         fun setStickerCallback(callback: (String) -> Unit) {
212             val plugin = PluginRegistry.getPlugin("send_sticker") as? SendStickerPlugin
213             plugin?.onStickerSent = callback
214         }
215
216         fun setImageGeneratedCallback(callback: (String) -> Unit) {
217             _generateImagePlugin?.onImageGenerated = callback
218         }
219
220         fun setAssociatedEventId(eventId: String?) {
221             _generateImagePlugin?.associatedEventId = eventId
222         }
223
224         fun setImagePluginWorldId(worldId: String) {
225             _generateImagePlugin?.worldId = worldId
226         }
227
228         fun isInitialized(): Boolean = settingsManager != null
229     }
230     /** 应用入口: Application生命周期管理, 全局异常捕获和Toast提示 */
231     package com.aicompanion
232
233     import android.app.Application
234     import android.util.Log
235     import android.widget.Toast
236     import com.aicompanion.AppContainer
237     import com.aicompanion.theme.ThemeManager
238
239     class CompanionApp : Application() {
240         override fun onCreate() {
241             super.onCreate()
242             com.aicompanion.util.AppLogger.init(this)
243             com.aicompanion.rag.RagConfig.init(this)  // 初始化RAG配置持久化
244             AppContainer.initialize(this)
245             com.aicompanion.migration.DataMigrationManager.migrateIfNeeded(this)
246
247             // 初始化外观模式 (DayNight 暗色/亮色/跟随系统)
248             ThemeManager.initAppearance(this)
```

```
249
250         // 注册全局主题观察器 — 每个 Activity 自动应用配色方案
251         ThemeManager.registerGlobalObserver(this)
252
253         // 注册 ThemeState → ThemeManager 同步桥接
254         // 当 Compose UI 切换主题时，自动同步到旧系统的 SharedPreferences
255         com.aicompanion.theme.ThemeState.addListener { themeId, darkMode ->
256             // 将 ThemeState 的变更写回 ThemeManager 使用的 SharedPreferences 格式
257             val prefs = getSharedPreferences("app_prefs", MODE_PRIVATE)
258             prefs.edit().putString("theme_scheme", themeId.key).apply()
259
260             val appearanceValue = when (darkMode) {
261                 true -> "dark"
262                 false -> "light"
263                 else -> "system"
264             }
265             prefs.edit().putString("appearance_mode", appearanceValue).apply()
266
267             // 同时更新 ThemeManager 的外观模式（立即生效）
268             try {
269                 val mode = when (darkMode) {
270                     true -> com.aicompanion.theme.AppearanceMode.DARK
271                     false -> com.aicompanion.theme.AppearanceMode.LIGHT
272                     else -> com.aicompanion.theme.AppearanceMode.SYSTEM
273                 }
274                 ThemeManager.applyAppearanceMode(mode)
275             } catch (_: Exception) {}
276         }
277
278         Thread.setDefaultUncaughtExceptionHandler { thread, throwable ->
279             com.aicompanion.util.AppLogger.e("CompanionApp", "FATAL on ${thread.name}: ${throwable.javaClass.simpleName}: ${throwable.
280             com.aicompanion.util.AppLogger.flush()
281             try {
282                 Toast.makeText(this, "应用发生错误，请查看日志", Toast.LENGTH_LONG).show()
283             } catch (_: Exception) {}
284         }
285     }
286 }
287 package com.aicompanion.action
288
289 import android.app.AlarmManager
290 import android.app.Notification
291 import android.app.NotificationChannel
292 import android.app.NotificationManager
293 import android.app.PendingIntent
294 import android.content.BroadcastReceiver
295 import android.content.Context
296 import android.content.Intent
297 import android.os.Build
298 import com.aicompanion.models.ToolDefinition
299 import com.aicompanion.plugin.PluginRegistry
300 import com.aicompanion.util.AppLogger
301 import org.json.JSONObject
302 import java.text.SimpleDateFormat
303 import java.util.Calendar
304 import java.util.Locale
305
306 class AIActionManager(private val context: Context) {
307
308     companion object {
309         private const val TAG = "AIActionManager"
```

```
        const val CHANNEL_REMINDER = "reminder_channel"
        const val REQUEST_ALARM = 5001
        const val REQUEST_SCHEDULE = 6001
        const val ACTION_ALARM_TRIGGER = "com.aicompanion.action.ALARM_TRIGGER"
        const val ACTION_SCHEDULE_TRIGGER = "com.aicompanion.action.SCHEDULE_TRIGGER"
        const val EXTRA_TITLE = "extra_title"
        const val EXTRA_MESSAGE = "extra_message"
        const val EXTRA_ID = "extra_id"
    }

    private val prefs = context.getSharedPreferences("schedule_data", Context.MODE_PRIVATE)

    init {
        createNotificationChannels()
    }

    private fun createNotificationChannels() {
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
            val manager = context.getSystemService(NotificationManager::class.java)
            val channel = NotificationChannel(
                CHANNEL_REMINDER, "提醒通知",
                NotificationManager.IMPORTANCE_HIGH
            ).apply {
                description = "闹钟和日程提醒"
                enableVibration(true)
                vibrationPattern = longArrayOf(0, 500, 500, 500)
                setSound(
                    android.media.RingtoneManager.getDefaultUri(android.media.RingtoneManager.TYPE_ALARM),
                    android.media.AudioAttributes.Builder()
                        .setUsage(android.media.AudioAttributes.USAGE_ALARM)
                        .setContentType(android.media.AudioAttributes.CONTENT_TYPE_SONIFICATION)
                        .build()
                )
            }
            manager.createNotificationChannel(channel)
        }
    }

    fun getToolDefinitions(): List<ToolDefinition> {
        return PluginRegistry.getEnabledDefinitions()
    }

    suspend fun executeTool(name: String, arguments: String): String {
        return try {
            PluginRegistry.executePlugin(name, arguments)
        } catch (e: Exception) {
            AppLogger.e(TAG, "executeTool $name failed: ${e.message}")
            "工具执行失败: ${e.message}"
        }
    }

    fun setAlarm(info: AlarmInfo): String {
        val cal = Calendar.getInstance().apply {
            set(Calendar.HOUR_OF_DAY, info.hour)
            set(Calendar.MINUTE, info.minute)
            set(Calendar.SECOND, 0)
            set(Calendar.MILLISECOND, 0)
            if (before(Calendar.getInstance())) {
                add(Calendar.DAY_OF_YEAR, 1)
            }
        }

```

```
372         val id = prefs.getInt("last_alarm_id", REQUEST_ALARM) + 1
373         prefs.edit().putInt("last_alarm_id", id).apply()
374
375         val intent = Intent(context, ReminderReceiver::class.java).apply {
376             action = ACTION_ALARM_TRIGGER
377             putExtra(EXTRA_TITLE, "\u23F0 ${info.label}")
378             putExtra(EXTRA_MESSAGE, "${info.hour}：${String.format("%02d", info.minute)} 的闹钟响了！")
379             putExtra(EXTRA_ID, id)
380         }
381
382         val pendingIntent = PendingIntent.getBroadcast(
383             context, id, intent,
384             PendingIntent.FLAG_UPDATE_CURRENT or PendingIntent.FLAG_IMMUTABLE
385         )
386
387         val alarmManager = context.getSystemService(Context.ALARM_SERVICE) as AlarmManager
388         if (android.os.Build.VERSION.SDK_INT >= android.os.Build.VERSION_CODES.S) {
389             if (alarmManager.canScheduleExactAlarms()) {
390                 alarmManager.setExactAndAllowWhileIdle(AlarmManager.RTC_WAKEUP, cal.timeInMillis, pendingIntent)
391             } else {
392                 alarmManager.setAndAllowWhileIdle(AlarmManager.RTC_WAKEUP, cal.timeInMillis, pendingIntent)
393             }
394         } else {
395             alarmManager.setExactAndAllowWhileIdle(AlarmManager.RTC_WAKEUP, cal.timeInMillis, pendingIntent)
396         }
397
398         val timeStr = "${info.hour}:${String.format("%02d", info.minute)}"
399         val triggerAt = SimpleDateFormat("yyyy-MM-dd HH:mm:ss", java.util.Locale.getDefault()).format(cal.time)
400         AppLogger.d(TAG, "setAlarm: id=$id, label=${info.label}, trigger=$triggerAt, delayMs=${cal.timeInMillis - System.currentTimeMillis()}"
401         return "闹钟已设置：${info.label}将在 $timeStr 触发"
402     }
403
404     fun addSchedule(info: ScheduleInfo): String {
405         val id = prefs.getInt("last_schedule_id", REQUEST_SCHEDULE) + 1
406         prefs.edit().putInt("last_schedule_id", id).apply()
407
408         val cal = Calendar.getInstance().apply {
409             set(Calendar.YEAR, info.year)
410             set(Calendar.MONTH, info.month)
411             set(Calendar.DAY_OF_MONTH, info.dayOfMonth)
412             set(Calendar.HOUR_OF_DAY, info.hour)
413             set(Calendar.MINUTE, info.minute)
414             set(Calendar.SECOND, 0)
415             set(Calendar.MILLISECOND, 0)
416         }
417
418         val scheduleKey = "schedule_$id"
419         val scheduleJson = JSONObject().apply {
420             put("id", id)
421             put("description", info.description)
422             put("year", info.year)
423             put("month", info.month)
424             put("day", info.dayOfMonth)
425             put("hour", info.hour)
426             put("minute", info.minute)
427             put("timestamp", cal.timeInMillis)
428         }
429         prefs.edit().putString(scheduleKey, scheduleJson.toString()).commit()
430
431         val intent = Intent(context, ReminderReceiver::class.java).apply {
432             action = ACTION_SCHEDULE_TRIGGER
433             putExtra(EXTRA_TITLE, "\uD83D\uDCC5 日程提醒")
```

```
434             putExtra(EXTRA_MESSAGE, info.description)
435             putExtra(EXTRA_ID, id)
436         }
437
438         val pendingIntent = PendingIntent.getBroadcast(
439             context, id + 1000, intent,
440             PendingIntent.FLAG_UPDATE_CURRENT or PendingIntent.FLAG_IMMUTABLE
441         )
442
443         val alarmManager = context.getSystemService(Context.ALARM_SERVICE) as AlarmManager
444         if (android.os.Build.VERSION.SDK_INT >= android.os.Build.VERSION_CODES.S) {
445             if (alarmManager.canScheduleExactAlarms()) {
446                 alarmManager.setExactAndAllowWhileIdle(AlarmManager.RTC_WAKEUP, cal.timeInMillis, pendingIntent)
447             } else {
448                 alarmManager.setAndAllowWhileIdle(AlarmManager.RTC_WAKEUP, cal.timeInMillis, pendingIntent)
449             }
450         } else {
451             alarmManager.setExactAndAllowWhileIdle(AlarmManager.RTC_WAKEUP, cal.timeInMillis, pendingIntent)
452         }
453
454         val dateStr = SimpleDateFormat("M月d日 HH:mm", Locale.getDefault()).format(cal.time)
455         return "日程已添加：${info.description}（$dateStr）"
456     }
457
458     fun getAllSchedules(): List<ScheduleInfo> {
459         val list = mutableListOf<ScheduleInfo>()
460         val allKeys = prefs.all.keys.filter { it.startsWith("schedule_") }
461         for (key in allKeys) {
462             try {
463                 val json = JSONObject(prefs.getString(key, "{}"))
464                 list.add(ScheduleInfo(
465                     hour = json.optInt("hour"),
466                     minute = json.optInt("minute"),
467                     dayOfMonth = json.optInt("day"),
468                     month = json.optInt("month"),
469                     year = json.optInt("year"),
470                     description = json.optString("description", "")
471                 ))
472             } catch (_: Exception) {}
473         }
474         return list
475     }
476
477     data class AlarmInfo(
478         val hour: Int,
479         val minute: Int,
480         val label: String = "闹钟",
481         val isDelay: Boolean = false
482     )
483
484     data class ScheduleInfo(
485         val hour: Int,
486         val minute: Int,
487         val dayOfMonth: Int,
488         val month: Int,
489         val year: Int,
490         val description: String
491     )
492
493     class ReminderReceiver : BroadcastReceiver() {
494         override fun onReceive(context: Context, intent: Intent) {
495             val title = intent.getStringExtra(EXTRA_TITLE) ?: "提醒"
```

```
496             val message = intent.getStringExtra(EXTRA_MESSAGE) ?: ""
497             val id = intent.getIntExtra(EXTRA_ID, 0)
498             AppLogger.d(TAG, "ReminderReceiver: ALARM FIRED! id=$id, title=$title, action=${intent.action}")
499
500             val alarmIntent = Intent(context, com.aicompanion.ui.AlarmActivity::class.java).apply {
501                 flags = Intent.FLAG_ACTIVITY_NEW_TASK or Intent.FLAG_ACTIVITY_CLEAR_TOP
502                 putExtra("alarm_label", title)
503                 putExtra("alarm_message", message)
504             }
505
506             val fullScreenPendingIntent = PendingIntent.getActivity(
507                 context, id, alarmIntent,
508                 PendingIntent.FLAG_UPDATE_CURRENT or PendingIntent.FLAG_IMMUTABLE
509             )
510
511             val openIntent = context.packageManager
512                 .getLaunchIntentForPackage(context.packageName)?.apply {
513                     flags = Intent.FLAG_ACTIVITY_NEW_TASK or Intent.FLAG_ACTIVITY_CLEAR_TOP
514                 }
515
516             val contentPendingIntent = PendingIntent.getActivity(
517                 context, id + 10000, openIntent,
518                 PendingIntent.FLAG_UPDATE_CURRENT or PendingIntent.FLAG_IMMUTABLE
519             )
520
521             val builder = if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
522                 Notification.Builder(context, CHANNEL_REMINDER)
523             } else {
524                 @Suppress("DEPRECATION")
525                 Notification.Builder(context)
526             }
527
528             val notification = builder
529                 .setContentTitle(title)
530                 .setContentText(message)
531                 .setSmallIcon(android.R.drawable.ic_dialog_info)
532                 .setAutoCancel(true)
533                 .setContentIntent(contentPendingIntent)
534                 .setFullScreenIntent(fullScreenPendingIntent, true)
535                 .setPriority(Notification.PRIORITY_MAX)
536                 .setDefaults(Notification.DEFAULT_ALL)
537                 .setCategory(Notification.CATEGORY_ALARM)
538                 .build()
539             notification.flags = notification.flags or Notification.FLAG_INSISTENT or Notification.FLAG_NO_CLEAR
540
541             val manager = context.getSystemService(Context.NOTIFICATION_SERVICE) as NotificationManager
542             manager.notify(id, notification)
543
544             try {
545                 if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.Q) {
546                     val fullScreenIntent = Intent(context, com.aicompanion.ui.AlarmActivity::class.java).apply {
547                         flags = Intent.FLAG_ACTIVITY_NEW_TASK or Intent.FLAG_ACTIVITY_CLEAR_TOP
548                         putExtra("alarm_label", title)
549                         putExtra("alarm_message", message)
550                     }
551                     context.startActivity(fullScreenIntent)
552                 }
553             } catch (_: Exception) {}
554         }
555     }
556 }
```

```
/** 好感度管理器: 根据用户消息数量/心情/互动频率计算好感度, 行为评估影响好感度变化 */
package com.aicompanion.affection

import android.content.Context
import android.content.SharedPreferences
import com.aicompanion.models.Emotion
import com.aicompanion.util.AppLogger

data class DailyStats(
    val messagesToday: Int = 0,
    val affectionChange: Int = 0
)

class AffectionManager(private val context: Context, private val personaId: String = "default") {

    private val prefs: SharedPreferences = context.getSharedPreferences(
        "affection_data_$personaId",
        Context.MODE_PRIVATE
    )

    var affectionLevel: Int
        get() = prefs.getInt("affection_level", 50)
        private set(value) {
            prefs.edit().putInt("affection_level", value.coerceIn(0, 100)).apply()
        }

    var totalInteractionDays: Int
        get() = prefs.getInt("total_days", 0)
        private set(value) {
            prefs.edit().putInt("total_days", value).apply()
        }

    var firstUseDate: Long
        get() = prefs.getLong("first_use_date", 0L)
        private set(value) {
            prefs.edit().putLong("first_use_date", value).apply()
        }

    var lastActiveDate: String
        get() = prefs.getString("last_active_date", "") ?: ""
        private set(value) {
            prefs.edit().putString("last_active_date", value).apply()
        }

    var messagesToday: Int
        get() = prefs.getInt("messages_today", 0)
        private set(value) {
            prefs.edit().putInt("messages_today", value).apply()
        }

    var affectionChangeToday: Int
        get() = prefs.getInt("affection_change_today", 0)
        private set(value) {
            prefs.edit().putInt("affection_change_today", value).apply()
        }

    init {
        if (firstUseDate == 0L) {
            firstUseDate = System.currentTimeMillis()
        }
        updateDailyStats()
    }
```

```
619
620     fun updateDailyStats() {
621         val today = getTodayString()
622         if (lastActiveDate != today) {
623             lastActiveDate = today
624             totalInteractionDays += 1
625             messagesToday = 0
626             affectionChangeToday = 0
627             addAffection(1)
628         }
629     }
630
631     fun addMessage() {
632         messagesToday += 1
633     }
634
635     fun addAffection(amount: Int, reason: String = "") {
636         val current = affectionLevel
637         val newValue = (current + amount).coerceIn(0, 100)
638         affectionLevel = newValue
639         AppLogger.w("Affection", "addAffection: +$amount -> $newValue")
640
641         prefs.edit().putString("last_affection_change", "$reason: $current -> $newValue").apply()
642     }
643
644     fun shouldTriggerPersonalityEvolution(): Boolean {
645         val personaPrefs = context.getSharedPreferences("persona_data_$personaId", Context.MODE_PRIVATE)
646         val enabled = personaPrefs.getBoolean("personality_evolution_enabled", true)
647         if (!enabled) return false
648
649         val lastEvolution = personaPrefs.getInt("last_evolution_affection", 50)
650         val current = affectionLevel
651         if (current >= lastEvolution + 5) {
652             personaPrefs.edit().putInt("last_evolution_affection", current).apply()
653             //AppLogger.i("Affection", "personalityEvolution triggered: affection=$current")
654             return true
655         }
656         return false
657     }
658
659     fun shouldTriggerPersonalitySummary(): Boolean {
660         val globalPrefs = context.getSharedPreferences("companion_settings", Context.MODE_PRIVATE)
661         val userDef = globalPrefs.getString("user_personality_def", "") ?: ""
662         if (userDef.isNotBlank()) return false
663
664         val lastSummary = globalPrefs.getInt("last_personality_summary_affection", 0)
665         val current = affectionLevel
666         if (current >= lastSummary + 10) {
667             globalPrefs.edit().putInt("last_personality_summary_affection", current).apply()
668             //AppLogger.i("Affection", "personalitySummary triggered: affection=$current")
669             return true
670         }
671         return false
672     }
673
674     fun decreaseAffection(amount: Int, reason: String = "") {
675         val current = affectionLevel
676         if (current >= 90) {
677             return
678         }
679         val newValue = (current - amount).coerceIn(0, 100)
680         affectionLevel = newValue
```

```
        AppLogger.w("Affection", "decreaseAffection: -$amount -> $newValue")

        prefs.edit().putString("last_affection_change", "$reason: $current -> $newValue").apply()
    }

    fun evaluateUserBehavior(message: String, emotion: Emotion, isOffensive: Boolean = false) {
        if (isOffensive) {
            decreaseAffection(2, "用户说了冒犯的话")
            AppLogger.w("Affection", "evaluateBehavior: offensive, emotion=$emotion")
            return
        }

        when (emotion) {
            Emotion.HAPPY -> addAffection(1, "用户心情好")
            Emotion.SAD -> {}
            Emotion.ANGRY -> decreaseAffection(1, "用户对AI生气")
            Emotion.TSUNDERE -> addAffection(1, "傲娇互动")
            Emotion.SURPRISED -> {}
            Emotion.NEUTRAL -> {}
        }

        val positiveWords = listOf("谢谢", "爱你", "可爱", "棒", "喜欢", "好乖")
        val negativeWords = listOf("滚", "烦", "笨", "傻", "讨厌", "闭嘴", "别吵")

        val lowerMsg = message.lowercase()
        if (positiveWords.any { lowerMsg.contains(it) }) {
            addAffection(1, "用户说了好话")
        }
        if (negativeWords.any { lowerMsg.contains(it) }) {
            decreaseAffection(2, "用户说了不好的话")
        }
        AppLogger.w("Affection", "evaluateBehavior: emotion=$emotion, affection=$affectionLevel")
    }

    fun getAffectionTitle(): String {
        return when (affectionLevel) {
            in 0..20 -> "陌生人"
            in 21..40 -> "认识的人"
            in 41..60 -> "朋友"
            in 61..80 -> "好朋友"
            in 81..95 -> "亲密伙伴"
            in 96..100 -> "最重要的人"
            else -> "未知"
        }
    }

    fun getAffectionColor(): Int {
        return when (affectionLevel) {
            in 0..30 -> android.graphics.Color.parseColor("#f44336")
            in 31..60 -> android.graphics.Color.parseColor("#ff9800")
            in 61..80 -> android.graphics.Color.parseColor("#4caf50")
            in 81..100 -> android.graphics.Color.parseColor("#e91e63")
            else -> android.graphics.Color.GRAY
        }
    }

    fun getDaysSinceFirstUse(): Int {
        val diff = System.currentTimeMillis() - firstUseDate
        return (diff / (1000 * 60 * 60 * 24)).toInt() + 1
    }

    fun getDailyStats(): DailyStats {
```

```
            return DailyStats(messagesToday = messagesToday, affectionChange = affectionChangeToday)
        }

        private fun getTodayString(): String {
            val sdf = java.text.SimpleDateFormat("yyyy-MM-dd", java.util.Locale.getDefault())
            return sdf.format(java.util.Date())
        }
    }
package com.aicompanion.album

import android.content.Intent
import android.os.Bundle
import android.os.Handler
import android.os.Looper
import android.view.GestureDetector
import android.view.Gravity
import android.view.MotionEvent
import android.view.ScaleGestureDetector
import android.view.View
import android.widget.EditText
import android.widget.FrameLayout
import android.widget.GridLayout
import android.widget.ImageView
import android.widget.LinearLayout
import android.widget.ProgressBar
import android.widget.ScrollView
import android.widget.TextView
import android.widget.Toast
import androidx.appcompat.app.AppCompatActivity
import androidx.core.content.FileProvider
import androidx.lifecycle.lifecycleScope
import com.aicompanion.R
import com.aicompanion.theme.ThemeManager
import kotlinx.coroutines.launch
import java.io.File

class MemorialAlbumActivity : AppCompatActivity() {

    private lateinit var gridContainer: LinearLayout
    private lateinit var progressBar: ProgressBar
    private lateinit var emptyView: LinearLayout
    private lateinit var scrollView: ScrollView
    private var currentTemplateIndex = 0
    private val templateChips = mutableListOf<com.google.android.material.chip.Chip>()

    private val slideshowHandler = Handler(Looper.getMainLooper())
    private var slideshowRunnable: Runnable? = null
    private var slideshowDialog: android.app.Dialog? = null
    private var slideshowIndex = 0
    private var allEntries: List<AlbumEntry> = emptyList()

    private var fullScreenDialog: android.app.Dialog? = null

    private var charRefDialog: android.app.AlertDialog? = null
    private var charRefTempPath: String? = null

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        currentTemplateIndex = MemorialAlbumManager.getCurrentTemplateIndex(this)

```

```
804         val scheme = ThemeManager.getCurrentScheme(this)
805         val bgColor = parseColor(scheme.backgroundDark, "#0a0a1a")
806         val cardColor = parseColor(scheme.cardColor, "#1a1a30")
807         val primaryColor = parseColor(scheme.primaryColor, "#667eea")
808         val textColor = parseColor(scheme.textColor, "#ececf4")
809         val textSecColor = parseColor(scheme.textSecondaryColor, "#99aabb")
810
811         window.statusBarColor = bgColor
812
813         val root = LinearLayout(this).apply {
814             orientation = LinearLayout.VERTICAL
815             setBackgroundColor(bgColor)
816         }
817
818         val toolbar = com.google.android.material.appbar.MaterialToolbar(this).apply {
819             title = "纪念相册"
820             setNavigationIcon(android.R.drawable.ic_menu_revert)
821             setNavigationOnClickListener { finish() }
822             setTitleTextColor(textColor)
823             setBackgroundColor(bgColor)
824         }
825         root.addView(toolbar)
826
827         progressBar = ProgressBar(this).apply {
828             visibility = View.GONE
829             layoutParams = LinearLayout.LayoutParams(
830                 LinearLayout.LayoutParams.WRAP_CONTENT, LinearLayout.LayoutParams.WRAP_CONTENT
831             ).apply { gravity = Gravity.CENTER }
832         }
833         root.addView(progressBar)
834
835         if (!MemorialAlbumManager.isImageModelConfigured(this)) {
836             val warningCard = LinearLayout(this).apply {
837                 orientation = LinearLayout.VERTICAL
838                 val pad = dp(14); setPadding(pad, dp(10), pad, dp(10))
839                 layoutParams = llMatchWrap().apply { setMargins(dp(12), dp(8), dp(12), 0) }
840                 background = android.graphics.drawable.GradientDrawable().apply {
841                     setColor(0xFF221a1a.toInt())
842                     cornerRadius = dp(12).toFloat()
843                     setStroke(dp(1), 0xFF4a2a2a.toInt())
844                 }
845             }
846             warningCard.addView(TextView(this).apply {
847                 text = "□□ 图片生成模型未配置\n请前往 设置 → AI功能 → 图片生成配置 中填写API地址和密钥"
848                 textSize = 13f; setTextColor(0xFFff8866.toInt())
849             })
850             root.addView(warningCard)
851         }
852
853         val actionRow = LinearLayout(this).apply {
854             orientation = LinearLayout.HORIZONTAL
855             gravity = Gravity.CENTER_VERTICAL
856             val pad = dp(12); setPadding(pad, dp(6), pad, dp(6))
857         }
858
859         val btnGenerate = com.google.android.material.button.MaterialButton(this).apply {
860             text = "□ 随机生成"
861             textSize = 13f; setTextColor(android.graphics.Color.WHITE)
862             backgroundTintList = android.content.res.ColorStateList.valueOf(primaryColor)
863             cornerRadius = 20
864             layoutParams = LinearLayout.LayoutParams(LinearLayout.LayoutParams.WRAP_CONTENT, LinearLayout.LayoutParams.WRAP_CONT
865             setOnClickListener { generateRandomImage() }
```

```
866         }
867         actionRow.addView(btnGenerate)
868
869         actionRow.addView(View(this).apply { layoutParams = LinearLayout.LayoutParams(dp(8), 1) })
870
871         val btnGenChar = com.google.android.material.button.MaterialButton(this).apply {
872             text = "□ 角色形象"
873             textSize = 13f; setTextColor(android.graphics.Color.WHITE)
874             backgroundTintList = android.content.res.ColorStateList.valueOf(0xFF9C27B0.toInt())
875             cornerRadius = 20
876             layoutParams = LinearLayout.LayoutParams(LinearLayout.LayoutParams.WRAP_CONTENT, LinearLayout.LayoutParams.WRAP_CONT
877             setOnClickListener { showCharacterRefDialog() }
878         }
879         actionRow.addView(btnGenChar)
880
881         actionRow.addView(View(this).apply { layoutParams = LinearLayout.LayoutParams(dp(8), 1) })
882
883         val btnSlideshow = com.google.android.material.button.MaterialButton(this).apply {
884             text = "□ 幻灯片"
885             textSize = 13f; setTextColor(android.graphics.Color.WHITE)
886             backgroundTintList = android.content.res.ColorStateList.valueOf(0xFF00897B.toInt())
887             cornerRadius = 20
888             layoutParams = LinearLayout.LayoutParams(LinearLayout.LayoutParams.WRAP_CONTENT, LinearLayout.LayoutParams.WRAP_CONT
889             setOnClickListener { startSlideshow() }
890         }
891         actionRow.addView(btnSlideshow)
892
893         actionRow.addView(View(this).apply { layoutParams = LinearLayout.LayoutParams(dp(8), 1) })
894
895         val captionHint = TextView(this).apply {
896             text = "点击图片查看大图"
897             textSize = 11f; setTextColor(textSecColor)
898             layoutParams = LinearLayout.LayoutParams(0, LinearLayout.LayoutParams.WRAP_CONTENT, 1f)
899         }
900         actionRow.addView(captionHint)
901         root.addView(actionRow)
902
903         val templateScroll = android.widget.HorizontalScrollView(this).apply {
904             isHorizontalScrollBarEnabled = false
905             layoutParams = LinearLayout.LayoutParams(LinearLayout.LayoutParams.MATCH_PARENT, LinearLayout.LayoutParams.WRAP_CONT
906             val pad = dp(8); setPadding(pad, dp(2), pad, dp(2))
907         }
908         val templateRow = LinearLayout(this).apply { orientation = LinearLayout.HORIZONTAL }
909         val templates = MemorialAlbumManager.layoutTemplates
910         for ((i, tmpl) in templates.withIndex()) {
911             val chip = com.google.android.material.chip.Chip(this).apply {
912                 text = "${tmpl.icon} ${tmpl.name}"
913                 textSize = 11f
914                 tag = i
915                 setOnClickListener {
916                     currentTemplateIndex = i
917                     MemorialAlbumManager.saveCurrentTemplateIndex(this@MemorialAlbumActivity, i)
918                     updateTemplateChipStyles()
919                     loadAlbum()
920                 }
921             }
922             templateChips.add(chip)
923             templateRow.addView(chip)
924         }
925         templateScroll.addView(templateRow)
926         root.addView(templateScroll)
927         updateTemplateChipStyles()
```

```
928
929         if (MemorialAlbumManager.hasCharacterRefImage(this)) {
930             val refPath = MemorialAlbumManager.getCharacterRefImagePath(this)
931             val refFile = File(refPath)
932             if (refFile.exists()) {
933                 val refPreviewRow = LinearLayout(this).apply {
934                     orientation = LinearLayout.HORIZONTAL
935                     gravity = Gravity.CENTER_VERTICAL
936                     val pad = dp(12); setPadding(pad, dp(4), pad, dp(4))
937                 }
938                 val refThumbSize = dp(40)
939                 val refImageView = ImageView(this).apply {
940                     layoutParams = LinearLayout.LayoutParams(refThumbSize, refThumbSize)
941                     scaleType = ImageView.ScaleType.CENTER_CROP
942                     background = android.graphics.drawable.GradientDrawable().apply {
943                         setColor(0xFF2a2a4a.toInt())
944                         cornerRadius = dp(6).toFloat()
945                     }
946                     setOnClickListener { showCharacterRefDialog() }
947                 }
948                 val opts = android.graphics.BitmapFactory.Options().apply { inJustDecodeBounds = true }
949                 android.graphics.BitmapFactory.decodeFile(refPath, opts)
950                 opts.inSampleSize = calculateInSampleSize(opts, refThumbSize, refThumbSize)
951                 opts.inJustDecodeBounds = false
952                 val bitmap = android.graphics.BitmapFactory.decodeFile(refPath, opts)
953                 if (bitmap != null) refImageView.setImageBitmap(bitmap)
954                 refPreviewRow.addView(refImageView)
955                 refPreviewRow.addView(TextView(this).apply {
956                     text = "  角色形象已设定"
957                     textSize = 11f; setTextColor(0xFFc4b5fd.toInt())
958                     layoutParams = LinearLayout.LayoutParams(LinearLayout.LayoutParams.WRAP_CONTENT, LinearLayout.LayoutParams.WRAP_
959                 })
960                 root.addView(refPreviewRow)
961             }
962         }
963
964         val chipScroll = android.widget.HorizontalScrollView(this).apply {
965             isHorizontalScrollBarEnabled = false
966             layoutParams = LinearLayout.LayoutParams(LinearLayout.LayoutParams.MATCH_PARENT, LinearLayout.LayoutParams.WRAP_CONT
967             val pad = dp(8); setPadding(pad, 0, pad, dp(6))
968         }
969         val chipRow = LinearLayout(this).apply { orientation = LinearLayout.HORIZONTAL }
970         for ((title, _) in MemorialAlbumManager.getBuiltinPrompts()) {
971             chipRow.addView(com.google.android.material.chip.Chip(this).apply {
972                 text = title; textSize = 11f
973                 chipBackgroundColor = android.content.res.ColorStateList.valueOf(cardColor)
974                 setTextColor(0xFFc4b5fd.toInt())
975                 setOnClickListener { generateImageForTitle(title) }
976             })
977         }
978         chipScroll.addView(chipRow)
979         root.addView(chipScroll)
980
981         emptyView = LinearLayout(this).apply {
982             orientation = LinearLayout.VERTICAL
983             gravity = Gravity.CENTER
984             layoutParams = LinearLayout.LayoutParams(LinearLayout.LayoutParams.MATCH_PARENT, LinearLayout.LayoutParams.WRAP_CONT
985                 setMargins(0, dp(40), 0, 0)
986             }
987         }
988         emptyView.addView(TextView(this).apply {
989             text = "□"
```

```
            textSize = 40f
            gravity = Gravity.CENTER
        })
        emptyView.addView(TextView(this).apply {
            text = "还没有纪念图片\n点击上方按钮生成吧~"
            textSize = 14f; setTextColor(textSecColor)
            gravity = Gravity.CENTER
            setPadding(0, dp(8), 0, 0)
        })
        root.addView(emptyView)

        scrollView = ScrollView(this).apply {
            layoutParams = LinearLayout.LayoutParams(LinearLayout.LayoutParams.MATCH_PARENT, 0, 1f)
        }
        gridContainer = LinearLayout(this).apply {
            orientation = LinearLayout.VERTICAL
            val pad = dp(8); setPadding(pad, 0, pad, dp(16))
        }
        scrollView.addView(gridContainer)
        root.addView(scrollView)

        setContentView(root)
        loadAlbum()
    }

    private fun updateTemplateChipStyles() {
        val scheme = ThemeManager.getCurrentScheme(this)
        val cardColor = parseColor(scheme.cardColor, "#1a1a30")
        val primaryColor = parseColor(scheme.primaryColor, "#667eea")
        for ((i, chip) in templateChips.withIndex()) {
            chip.chipBackgroundColor = android.content.res.ColorStateList.valueOf(
                if (i == currentTemplateIndex) primaryColor else cardColor
            )
            chip.setTextColor(
                if (i == currentTemplateIndex) android.graphics.Color.WHITE else 0xFFc4b5fd.toInt()
            )
        }
    }

    private fun formatMonthHeader(monthKey: String): String {
        val parts = monthKey.split("-")
        if (parts.size == 2) {
            val year = parts[0]
            val month = parts[1].toIntOrNull() ?: return monthKey
            return "${year}年${month}月"
        }
        return monthKey
    }

    private fun loadAlbum() {
        gridContainer.removeAllViews()
        val currentTemplate = MemorialAlbumManager.layoutTemplates[currentTemplateIndex]

        val entries = MemorialAlbumManager.getEntries(this)
            .sortedByDescending { it.createdAt }
        allEntries = entries

        emptyView.visibility = if (entries.isEmpty()) View.VISIBLE else View.GONE
        scrollView.visibility = if (entries.isEmpty()) View.GONE else View.VISIBLE

        val scheme = ThemeManager.getCurrentScheme(this)
```

```
1051        val cardColor = parseColor(scheme.cardColor, "#1a1a30")
1052        val textColor = parseColor(scheme.textColor, "#ececf4")
1053        val textSecColor = parseColor(scheme.textSecondaryColor, "#99aabb")
1054        val primaryColor = parseColor(scheme.primaryColor, "#667eea")
1055
1056        val screenWidth = resources.displayMetrics.widthPixels
1057        val colWidth = (screenWidth - dp(32)) / currentTemplate.columns
1058
1059        val grouped = entries.groupBy { it.createdAt.substring(0, 7) }
1060
1061        for ((monthKey, monthEntries) in grouped) {
1062            val monthHeader = TextView(this).apply {
1063                text = formatMonthHeader(monthKey)
1064                textSize = 14f
1065                setTextColor(primaryColor)
1066                setTypeface(null, android.graphics.Typeface.BOLD)
1067                setPadding(dp(4), dp(12), dp(4), dp(2))
1068                layoutParams = LinearLayout.LayoutParams(LinearLayout.LayoutParams.MATCH_PARENT, LinearLayout.LayoutParams.WRAP_CO
1069            }
1070            gridContainer.addView(monthHeader)
1071
1072            val divider = View(this).apply {
1073                layoutParams = LinearLayout.LayoutParams(LinearLayout.LayoutParams.MATCH_PARENT, dp(1)).apply {
1074                    setMargins(0, 0, 0, dp(8))
1075                }
1076                setBackgroundColor(0x22FFFFFF)
1077            }
1078            gridContainer.addView(divider)
1079
1080            val monthGrid = GridLayout(this).apply {
1081                columnCount = currentTemplate.columns
1082                val pad = dp(4); setPadding(pad, 0, pad, 0)
1083            }
1084
1085            for (entry in monthEntries) {
1086                val heightMultiplier = MemorialAlbumManager.aspectRatioToHeightMultiplier(entry.aspectRatio)
1087                val imageHeight = (colWidth * heightMultiplier).toInt()
1088
1089                val card = LinearLayout(this).apply {
1090                    orientation = LinearLayout.VERTICAL
1091                    background = android.graphics.drawable.GradientDrawable().apply {
1092                        setColor(cardColor)
1093                        cornerRadius = dp(12).toFloat()
1094                        setStroke(dp(1), parseColor(scheme.surfaceColor, "#2a2a4a"))
1095                    }
1096                    layoutParams = GridLayout.LayoutParams().apply {
1097                        width = colWidth
1098                        setMargins(dp(4), dp(4), dp(4), dp(4))
1099                        columnSpec = GridLayout.spec(GridLayout.UNDEFINED, 1f)
1100                    }
1101                }
1102
1103                val imageFrame = FrameLayout(this).apply {
1104                    layoutParams = LinearLayout.LayoutParams(LinearLayout.LayoutParams.MATCH_PARENT, imageHeight)
1105                    setBackgroundColor(cardColor)
1106                }
1107
1108                val imageFile = File(entry.imagePath)
1109                if (imageFile.exists()) {
1110                    val imageView = ImageView(this).apply {
1111                        layoutParams = FrameLayout.LayoutParams(FrameLayout.LayoutParams.MATCH_PARENT, FrameLayout.LayoutParams.MAT
1112                        scaleType = ImageView.ScaleType.FIT_CENTER
```

```
            }
            val opts = android.graphics.BitmapFactory.Options().apply { inJustDecodeBounds = true }
            android.graphics.BitmapFactory.decodeFile(imageFile.absolutePath, opts)
            opts.inSampleSize = calculateInSampleSize(opts, colWidth, imageHeight)
            opts.inJustDecodeBounds = false
            val bitmap = android.graphics.BitmapFactory.decodeFile(imageFile.absolutePath, opts)
            if (bitmap != null) imageView.setImageBitmap(bitmap)
            imageFrame.addView(imageView)
        }

        val captionOverlay = TextView(this).apply {
            text = entry.caption
            textSize = 11f; setTextColor(0xFFFFFFFF.toInt())
            maxLines = 2
            ellipsize = android.text.TextUtils.TruncateAt.END
            setPadding(dp(6), dp(4), dp(6), dp(4))
            visibility = if (entry.caption.isNotBlank()) View.VISIBLE else View.GONE
            layoutParams = FrameLayout.LayoutParams(FrameLayout.LayoutParams.MATCH_PARENT, FrameLayout.LayoutParams.WRAP_
                gravity = Gravity.BOTTOM
            }
            background = android.graphics.drawable.GradientDrawable().apply {
                setColor(0x88000000.toInt())
            }
        }
        imageFrame.addView(captionOverlay)
        imageFrame.setOnClickListener { showFullScreenViewer(entry.imagePath) }
        card.addView(imageFrame)

        val infoRow = LinearLayout(this).apply {
            orientation = LinearLayout.HORIZONTAL
            gravity = Gravity.CENTER_VERTICAL
            val pad = dp(8); setPadding(pad, dp(6), pad, dp(6))
        }

        infoRow.addView(TextView(this).apply {
            text = entry.title
            textSize = 12f; setTextColor(textColor)
            layoutParams = LinearLayout.LayoutParams(0, LinearLayout.LayoutParams.WRAP_CONTENT, 1f)
            maxLines = 1
            ellipsize = android.text.TextUtils.TruncateAt.END
        })

        infoRow.addView(TextView(this).apply {
            text = entry.createdAt.substring(5, 10)
            textSize = 10f; setTextColor(textSecColor)
        })
        card.addView(infoRow)

        if (entry.caption.isNotBlank()) {
            card.addView(TextView(this).apply {
                text = entry.caption
                textSize = 11f; setTextColor(textSecColor)
                maxLines = 2
                ellipsize = android.text.TextUtils.TruncateAt.END
                setPadding(dp(8), 0, dp(8), dp(6))
            })
        }

        card.setOnClickListener { showEntryDetail(entry) }

        monthGrid.addView(card)
    }
```

```
1175            gridContainer.addView(monthGrid)
1176        }
1177    }
1178
1179    private fun showFullScreenViewer(imagePath: String) {
1180        fullScreenDialog?.dismiss()
1181
1182        val dialog = android.app.Dialog(this, android.R.style.Theme_Black_NoTitleBar_Fullscreen)
1183        val imageView = ImageView(this).apply {
1184            scaleType = ImageView.ScaleType.FIT_CENTER
1185            layoutParams = FrameLayout.LayoutParams(FrameLayout.LayoutParams.MATCH_PARENT, FrameLayout.LayoutParams.MATCH_PAR
1186        }
1187
1188        val imageFile = File(imagePath)
1189        if (imageFile.exists()) {
1190            val opts = android.graphics.BitmapFactory.Options().apply { inJustDecodeBounds = true }
1191            android.graphics.BitmapFactory.decodeFile(imageFile.absolutePath, opts)
1192            opts.inSampleSize = calculateInSampleSize(opts, resources.displayMetrics.widthPixels, resources.displayMetrics.heightPixels)
1193            opts.inJustDecodeBounds = false
1194            val bitmap = android.graphics.BitmapFactory.decodeFile(imageFile.absolutePath, opts)
1195            if (bitmap != null) imageView.setImageBitmap(bitmap)
1196        }
1197
1198        val hintText = TextView(this).apply {
1199            text = "点击关闭 · 双指缩放 · 下滑退出"
1200            textSize = 12f
1201            setTextColor(0x88FFFFFF.toInt())
1202            gravity = Gravity.CENTER
1203            setPadding(0, dp(12), 0, dp(24))
1204            layoutParams = FrameLayout.LayoutParams(FrameLayout.LayoutParams.MATCH_PARENT, FrameLayout.LayoutParams.WRAP_CON
1205                gravity = Gravity.BOTTOM
1206            }
1207        }
1208
1209        val root = FrameLayout(this).apply {
1210            setBackgroundColor(android.graphics.Color.BLACK)
1211            addView(imageView)
1212            addView(hintText)
1213        }
1214
1215        var currentScale = 1f
1216        val scaleDetector = ScaleGestureDetector(this, object : ScaleGestureDetector.SimpleOnScaleGestureListener() {
1217            override fun onScale(detector: ScaleGestureDetector): Boolean {
1218                currentScale *= detector.scaleFactor
1219                currentScale = currentScale.coerceIn(0.5f, 5f)
1220                imageView.scaleX = currentScale
1221                imageView.scaleY = currentScale
1222                return true
1223            }
1224        })
1225
1226        val gestureDetector = GestureDetector(this, object : GestureDetector.SimpleOnGestureListener() {
1227            override fun onSingleTapConfirmed(e: MotionEvent): Boolean {
1228                dialog.dismiss()
1229                return true
1230            }
1231        })
1232
1233        var downY = 0f
1234        var isSwiping = false
1235
1236        root.setOnTouchListener { _, event ->
```

```
            scaleDetector.onTouchEvent(event)
            gestureDetector.onTouchEvent(event)
            when (event.actionMasked) {
                MotionEvent.ACTION_DOWN -> {
                    downY = event.rawY
                    isSwiping = false
                }
                MotionEvent.ACTION_MOVE -> {
                    if (!scaleDetector.isInProgress && currentScale < 1.1f) {
                        val dy = event.rawY - downY
                        if (dy > dp(10)) {
                            isSwiping = true
                            imageView.translationY = dy
                            imageView.alpha = 1f - (dy / resources.displayMetrics.heightPixels).coerceIn(0f, 1f)
                        }
                    }
                }
                MotionEvent.ACTION_UP, MotionEvent.ACTION_CANCEL -> {
                    if (isSwiping && imageView.translationY > dp(150)) {
                        dialog.dismiss()
                    } else {
                        imageView.animate().translationY(0f).alpha(1f).setDuration(200).start()
                    }
                    isSwiping = false
                }
            }
            true
        }

        dialog.setContentView(root)
        fullScreenDialog = dialog
        dialog.setOnDismissListener { fullScreenDialog = null }
        dialog.show()

        imageView.alpha = 0f
        imageView.animate().alpha(1f).setDuration(200).start()
        hintText.animate().alpha(0f).setStartDelay(3000).setDuration(500).start()
    }

    private fun shareImage(imagePath: String) {
        val file = File(imagePath)
        if (!file.exists()) {
            Toast.makeText(this, "图片文件不存在", Toast.LENGTH_SHORT).show()
            return
        }
        try {
            val uri = FileProvider.getUriForFile(this, "${packageName}.fileprovider", file)
            val intent = Intent(Intent.ACTION_SEND).apply {
                type = "image/*"
                putExtra(Intent.EXTRA_STREAM, uri)
                addFlags(Intent.FLAG_GRANT_READ_URI_PERMISSION)
            }
            startActivity(Intent.createChooser(intent, "分享图片"))
        } catch (e: Exception) {
            Toast.makeText(this, "分享失败: ${e.message}", Toast.LENGTH_SHORT).show()
        }
    }

    private fun showEntryDetail(entry: AlbumEntry) {
        val scheme = ThemeManager.getCurrentScheme(this)
        val textColor = parseColor(scheme.textColor, "#ececf4")
        val textSecColor = parseColor(scheme.textSecondaryColor, "#99aabb")
```

```
1299
1300        val contentView = LinearLayout(this).apply {
1301            orientation = LinearLayout.VERTICAL
1302            val pad = dp(16); setPadding(pad, pad, pad, pad)
1303        }
1304
1305        val imageFile = File(entry.imagePath)
1306        if (imageFile.exists()) {
1307            val imageView = ImageView(this).apply {
1308                layoutParams = LinearLayout.LayoutParams(LinearLayout.LayoutParams.MATCH_PARENT, LinearLayout.LayoutParams.WRAP_CO
1309                scaleType = ImageView.ScaleType.FIT_CENTER
1310                adjustViewBounds = true
1311            }
1312            val opts = android.graphics.BitmapFactory.Options().apply { inJustDecodeBounds = true }
1313            android.graphics.BitmapFactory.decodeFile(imageFile.absolutePath, opts)
1314            opts.inSampleSize = calculateInSampleSize(opts, 512, 512)
1315            opts.inJustDecodeBounds = false
1316            val bitmap = android.graphics.BitmapFactory.decodeFile(imageFile.absolutePath, opts)
1317            if (bitmap != null) imageView.setImageBitmap(bitmap)
1318            imageView.setOnClickListener { showFullScreenViewer(entry.imagePath) }
1319            contentView.addView(imageView)
1320        }
1321
1322        contentView.addView(TextView(this).apply {
1323            text = entry.title
1324            textSize = 16f; setTextColor(textColor)
1325            setTypeface(null, android.graphics.Typeface.BOLD)
1326            setPadding(0, dp(8), 0, dp(2))
1327        })
1328
1329        contentView.addView(TextView(this).apply {
1330            text = "${entry.createdAt} · ${entry.aspectRatio}"
1331            textSize = 12f; setTextColor(textSecColor)
1332        })
1333
1334        val captionLabel = TextView(this).apply {
1335            text = if (entry.caption.isNotBlank()) entry.caption else "点击添加文字注释..."
1336            textSize = 13f
1337            setTextColor(if (entry.caption.isNotBlank()) textColor else textSecColor)
1338            setPadding(0, dp(8), 0, dp(4))
1339        }
1340        contentView.addView(captionLabel)
1341
1342        contentView.addView(TextView(this).apply {
1343            text = "Prompt: ${entry.prompt.take(100)}"
1344            textSize = 10f; setTextColor(0xFF556677.toInt())
1345            setPadding(0, dp(4), 0, 0)
1346        })
1347
1348        val dialog = android.app.AlertDialog.Builder(this)
1349            .setView(contentView)
1350            .apply {
1351                setNeutralButton("分享") { _, _ ->
1352                    shareImage(entry.imagePath)
1353                }
1354                setNegativeButton("编辑注释") { _, _ ->
1355                    showCaptionEditor(entry)
1356                }
1357                setPositiveButton("删除") { _, _ ->
1358                    MemorialAlbumManager.deleteEntry(this@MemorialAlbumActivity, entry.id)
1359                    loadAlbum()
```

```
                Toast.makeText(this@MemorialAlbumActivity, "已删除", Toast.LENGTH_SHORT).show()
            }
        }
        .create()
    dialog.show()
    dialog.window?.setBackgroundDrawableResource(R.drawable.bg_settings_card)
}

private fun showCaptionEditor(entry: AlbumEntry) {
    val scheme = ThemeManager.getCurrentScheme(this)
    val textColor = parseColor(scheme.textColor, "#ececf4")

    val contentView = LinearLayout(this).apply {
        orientation = LinearLayout.VERTICAL
        val pad = dp(20); setPadding(pad, pad, pad, pad)
    }

    contentView.addView(TextView(this).apply {
        text = "编辑「${entry.title}」的文字注释"
        textSize = 14f; setTextColor(textColor)
        setPadding(0, 0, 0, dp(8))
    })

    val editText = EditText(this).apply {
        text = android.text.Editable.Factory.getInstance().newEditable(entry.caption)
        textSize = 14f; setTextColor(textColor)
        hint = "写下这一刻的故事..."
        setHintTextColor(0xFF667788.toInt())
        minLines = 3
        maxLines = 6
        gravity = Gravity.TOP
        setPadding(dp(12), dp(8), dp(12), dp(8))
        background = android.graphics.drawable.GradientDrawable().apply {
            setColor(0xFF1a1a38.toInt())
            cornerRadius = dp(8).toFloat()
            setStroke(dp(1), 0xFF2a2a4a.toInt())
        }
    }
    contentView.addView(editText)

    val dialog = android.app.AlertDialog.Builder(this)
        .setTitle("文字注释")
        .setView(contentView)
        .setPositiveButton("保存") { _, _ ->
            val newCaption = editText.text?.toString()?.trim() ?: ""
            MemorialAlbumManager.updateCaption(this, entry.id, newCaption)
            loadAlbum()
            Toast.makeText(this, "注释已保存", Toast.LENGTH_SHORT).show()
        }
        .setNegativeButton("取消", null)
        .create()
    dialog.show()
    dialog.window?.setBackgroundDrawableResource(R.drawable.bg_settings_card)
}

private fun generateRandomImage() {
    val (title, prompt) = MemorialAlbumManager.getRandomPrompt()
    doGenerate(title, prompt)
}

private fun generateImageForTitle(title: String) {
    MemorialAlbumManager.getBuiltinPrompts().find { it.first == title }?.let { doGenerate(it.first, it.second) }
```

```
    }

    private fun doGenerate(title: String, prompt: String) {
        if (!MemorialAlbumManager.isImageModelConfigured(this)) {
            Toast.makeText(this, "请先在设置中配置图片生成模型（API地址+密钥）", Toast.LENGTH_LONG).show()
            return
        }
        val currentTemplate = MemorialAlbumManager.layoutTemplates[currentTemplateIndex]
        val aspectRatio = currentTemplate.aspectRatio
        progressBar.visibility = View.VISIBLE
        Toast.makeText(this, "正在生成「$title」(${currentTemplate.name}, ${aspectRatio})...", Toast.LENGTH_SHORT).show()
        lifecycleScope.launch {
            val entry = MemorialAlbumManager.generateImage(this@MemorialAlbumActivity, prompt, title, aspectRatio = aspectRatio)
            progressBar.visibility = View.GONE
            if (entry != null) {
                Toast.makeText(this@MemorialAlbumActivity, "「$title」生成成功！", Toast.LENGTH_SHORT).show()
                loadAlbum()
            } else {
                Toast.makeText(this@MemorialAlbumActivity, "生成失败，请检查图片生成API配置", Toast.LENGTH_LONG).show()
            }
        }
    }

    private fun showCharacterRefDialog() {
        if (!MemorialAlbumManager.isImageModelConfigured(this)) {
            Toast.makeText(this, "请先在设置中配置图片生成模型（API地址+密钥）", Toast.LENGTH_LONG).show()
            return
        }

        val scheme = ThemeManager.getCurrentScheme(this)
        val textColor = parseColor(scheme.textColor, "#ececf4")
        val textSecColor = parseColor(scheme.textSecondaryColor, "#99aabb")

        val contentView = LinearLayout(this).apply {
            orientation = LinearLayout.VERTICAL
            gravity = Gravity.CENTER_HORIZONTAL
            val pad = dp(20); setPadding(pad, dp(16), pad, dp(16))
        }

        contentView.addView(TextView(this).apply {
            text = "角色参考图"
            textSize = 16f; setTextColor(textColor)
            setTypeface(null, android.graphics.Typeface.BOLD)
            setPadding(0, 0, 0, dp(8))
        })

        contentView.addView(TextView(this).apply {
            text = "生成角色和你的形象参考图，后续所有图片都会参考此形象保持一致性。"
            textSize = 12f; setTextColor(textSecColor)
            setPadding(0, 0, 0, dp(12))
        })

        val imageView = ImageView(this).apply {
            layoutParams = LinearLayout.LayoutParams(LinearLayout.LayoutParams.MATCH_PARENT, dp(300))
            scaleType = ImageView.ScaleType.FIT_CENTER
            background = android.graphics.drawable.GradientDrawable().apply {
                setColor(0xFF1a1a38.toInt())
                cornerRadius = dp(12).toFloat()
                setStroke(dp(1), 0xFF2a2a4a.toInt())
            }
        }
        contentView.addView(imageView)
```

```
1484
1485        val existingRefPath = MemorialAlbumManager.getCharacterRefImagePath(this)
1486        val existingFile = File(existingRefPath)
1487        if (existingFile.exists()) {
1488            val opts = android.graphics.BitmapFactory.Options().apply { inJustDecodeBounds = true }
1489            android.graphics.BitmapFactory.decodeFile(existingRefPath, opts)
1490            opts.inSampleSize = calculateInSampleSize(opts, 512, 512)
1491            opts.inJustDecodeBounds = false
1492            val bitmap = android.graphics.BitmapFactory.decodeFile(existingRefPath, opts)
1493            if (bitmap != null) imageView.setImageBitmap(bitmap)
1494            charRefTempPath = existingRefPath
1495        } else {
1496            imageView.setImageDrawable(null)
1497            charRefTempPath = null
1498        }
1499
1500        val statusText = TextView(this).apply {
```

```
92133                 } else {
92134                     listener?.onError("未识别到内容")
92135                 }
92136             }
92137             override fun onPartialResults(partialResults: Bundle?) {
92138                 val matches = partialResults?.getStringArrayList(SpeechRecognizer.RESULTS_RECOGNITION)
92139                 if (!matches.isNullOrEmpty()) {
92140                     listener?.onPartialResult(matches[0])
92141                 }
92142             }
92143             override fun onEvent(eventType: Int, params: Bundle?) {}
92144         })
92145 
92146         try {
92147             speechRecognizer?.startListening(intent)
92148         } catch (e: Exception) {
92149             AppLogger.e(TAG, "startListening: ${e.message}")
92150             listener?.onError("启动语音识别失败: ${e.message}")
92151             isListening = false
92152         }
92153     }
92154 
92155     private fun startSherpaListening() {
92156         AppLogger.w(TAG, "Sherpa-ONNX ASR 尚未集成，回退系统ASR")
92157         startSystemListening()
92158     }
92159 
92160     private fun checkSherpaAvailable(): Boolean {
92161         try {
92162             val modelDir = java.io.File(context.filesDir, "sherpa-onnx/models/sense-voice")
92163             return modelDir.exists() && modelDir.listFiles()?.isNotEmpty() == true
92164         } catch (_: Exception) {
92165             return false
92166         }
92167     }
92168 }
92169 /** 离线语音识别引擎: 语音转文字输入(离线模式) */
92170 package com.aicompanion.voice
92171 
92172 import android.content.Context
92173 
92174 class OfflineASREngine(context: Context) {
92175 
92176     fun initialize(): Boolean {
92177         return false
92178     }
92179 
92180     fun startListening(callback: (String) -> Unit) {
92181         callback("")
92182     }
92183 
92184     fun stopListening() {
92185     }
92186 
92187     fun release() {
92188     }
92189 }
92190 package com.aicompanion.voice
92191 
92192 import android.content.Context
92193 import android.media.MediaPlayer
92194 import com.aicompanion.models.Emotion
```

```
import com.aicompanion.network.ProviderAdapter
import com.aicompanion.settings.SettingsManager
import com.aicompanion.util.AppLogger
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.withContext
import okhttp3.MediaType.Companion.toMediaType
import okhttp3.MultipartBody
import okhttp3.Request
import okhttp3.RequestBody.Companion.toRequestBody
import org.json.JSONObject
import java.io.File
import java.util.UUID

data class TtsResult(
    val audioPath: String?,
    val audioUrl: String?,
    val success: Boolean,
    val error: String? = null
)

class TtsManager(private val context: Context) {

    companion object {
        private const val TAG = "TtsManager"
        const val ENGINE_CLOUD = "cloud"
        const val ENGINE_LOCAL = "local"
        const val ENGINE_AUTO = "auto"
        const val ENGINE_EDGE = "edge"
    }

    private val sm = SettingsManager(context)
    private var mediaPlayer: MediaPlayer? = null
    private var currentlyPlayingPath: String? = null
    private var onPlaybackCompleteListener: (() -> Unit)? = null

    val engineMode: String
        get() = sm.ttsEngineMode

    val isCloudConfigured: Boolean
        get() = sm.ttsApiUrl.isNotBlank() && sm.ttsApiKey.isNotBlank()

    suspend fun synthesize(text: String, emotion: Emotion = Emotion.NEUTRAL): TtsResult {
        return synthesizeWithVoice(text, null, emotion)
    }

    suspend fun synthesizeWithVoice(text: String, overrideVoice: String? = null, emotion: Emotion = Emotion.NEUTRAL): TtsResult {
        if (text.isBlank()) return TtsResult(null, null, false, "文本为空")

        val mode = engineMode
        if (mode == ENGINE_EDGE) {
            return try {
                val audioDir = File(context.filesDir, "tts_audio")
                val voiceId = overrideVoice ?: sm.ttsVoice.ifBlank { "zh-CN-XiaoxiaoNeural" }
                EdgeTtsEngine.synthesize(text, voiceId, audioDir, sm.ttsRate, sm.ttsPitch)
            } catch (e: Exception) {
                AppLogger.e(TAG, "Edge TTS失败: ${e.message}")
                TtsResult(null, null, false, "Edge TTS失败: ${e.message}")
            }
        }

        if (mode == ENGINE_LOCAL || (mode == ENGINE_AUTO && !isCloudConfigured)) {
            return TtsResult(null, null, true)
```

```
92257         }
92258
92259         return try {
92260             cloudSynthesize(text, overrideVoice)
92261         } catch (e: Exception) {
92262             AppLogger.e(TAG, "云端TTS失败，回退本地: ${e.message}")
92263             if (mode == ENGINE_AUTO) {
92264                 TtsResult(null, null, true)
92265             } else {
92266                 TtsResult(null, null, false, "云端TTS失败: ${e.message}")
92267             }
92268         }
92269     }
92270
92271     private suspend fun cloudSynthesize(text: String, overrideVoice: String? = null): TtsResult = withContext(Dispatchers.IO) {
92272         val ttsUrl = sm.ttsApiUrl
92273         val ttsKey = sm.ttsApiKey
92274         val ttsModel = sm.ttsModel
92275         var ttsVoice = overrideVoice ?: sm.ttsVoiceName.ifBlank { sm.ttsVoice }
92276         val formatType = com.aicompanion.settings.ServicePresets.findTtsPreset(sm.ttsProvider).formatType
92277         AppLogger.i(TAG, "TTS云端合成开始: text=${text.take(30)}, model=$ttsModel, voice=$ttsVoice, formatType=$formatType")
92278
92279         val isSiliconFlow = ttsUrl.contains("siliconflow", ignoreCase = true)
92280         val isCosyVoice = ttsModel.contains("CosyVoice", ignoreCase = true)
92281         if (isSiliconFlow && isCosyVoice && !ttsVoice.contains(":")) {
92282             val cosyVoiceNames = setOf("alex", "anna", "bella", "benjamin", "charles", "claire", "david", "diana")
92283             val voiceName = ttsVoice.lowercase()
92284             ttsVoice = if (voiceName in cosyVoiceNames) "$ttsModel:$voiceName" else "$ttsModel:alex"
92285         }
92286
92287         val jsonBody = ProviderAdapter.buildTtsRequest(formatType, ttsModel, text, ttsVoice)
92288         val headers = ProviderAdapter.buildTtsHeaders(formatType, ttsKey, ttsModel)
92289         val requestBuilder = Request.Builder()
92290             .url(ttsUrl)
92291             .post(jsonBody.toString().toRequestBody("application/json".toMediaType()))
92292         headers.forEach { (k, v) -> requestBuilder.addHeader(k, v) }
92293         val jsonRequest = requestBuilder.build()
92294
92295         val client = com.aicompanion.network.ApiClient.sharedClient
92296         client.newCall(jsonRequest).execute().use { jsonResp ->
92297
92298         if (jsonResp.code == 422 && formatType != com.aicompanion.network.ProviderAdapter.FORMAT_MIMO) {
92299             jsonResp.body?.close()
92300             AppLogger.w(TAG, "TTS请求422错误，可能是参数格式不匹配。常见原因: 1)voice名称不正确 2)模型不支持该voice 3)API格式不compatibl
92301             AppLogger.w(TAG, "TTS JSON格式422，尝试替换voice字段名")
92302
92303             val fallbackBody = org.json.JSONObject(jsonBody.toString()).apply {
92304                 if (has("voice")) {
92305                     val voiceVal = optString("voice", "")
92306                     put("voice_name", voiceVal)
92307                     remove("voice")
92308                 }
92309             }
92310             val fallbackRequest = Request.Builder()
92311                 .url(ttsUrl)
92312                 .addHeader("Authorization", "Bearer $ttsKey")
92313                 .addHeader("Content-Type", "application/json")
92314                 .post(fallbackBody.toString().toRequestBody("application/json".toMediaType()))
92315                 .build()
92316
92317             client.newCall(fallbackRequest).execute().use { fallbackResp ->
92318                 if (fallbackResp.isSuccessful) {
```

```
                return@withContext parseResponse(fallbackResp)
            }
            fallbackResp.body?.close()
            val errBody = fallbackResp.body?.string()?.take(300) ?: "无响应体"
            return@withContext TtsResult(null, null, false, "HTTP ${fallbackResp.code}: $errBody")
        }
    }

    if (!jsonResp.isSuccessful) {
        AppLogger.e(TAG, "TTS请求失败: HTTP ${jsonResp.code}, 常见原因: 1)API Key无效 2)余额不足 3)URL错误 4)模型名错误")
        val errBody = jsonResp.body?.string()?.take(300) ?: "无响应体"
        return@withContext TtsResult(null, null, false, "HTTP ${jsonResp.code}: $errBody")
    }

    return@withContext parseResponse(jsonResp, formatType)
    }
}

private fun parseResponse(response: okhttp3.Response, formatType: String = "openai"): TtsResult {
    if (!response.isSuccessful) {
        val errBody = response.body?.string()?.take(300) ?: "无响应体"
        return TtsResult(null, null, false, "HTTP ${response.code}: $errBody")
    }

    val contentType = response.header("Content-Type", "")
    val body = response.body ?: return TtsResult(null, null, false, "空响应")

    if (contentType?.contains("audio") == true || contentType?.contains("octet-stream") == true) {
        val extension = if (contentType.contains("wav")) "wav" else "mp3"
        val audioDir = File(context.filesDir, "tts_audio")
        if (!audioDir.exists()) audioDir.mkdirs()
        val audioFile = File(audioDir, "${System.currentTimeMillis()}_${UUID.randomUUID().toString().take(6)}.$extension")
        body.byteStream().use { input ->
            java.io.FileOutputStream(audioFile).use { output ->
                input.copyTo(output)
            }
        }
        return TtsResult(audioFile.absolutePath, null, true)
    }

    val bodyStr = body.string()
    try {
        // MiMoTTS 特殊解析：choices[0].message.audio.data (Base64)
        if (formatType == com.aicompanion.network.ProviderAdapter.FORMAT_MIMO) {
            val mimoB64 = com.aicompanion.network.ProviderAdapter.parseMimoResponse(bodyStr)
            if (!mimoB64.isNullOrBlank()) {
                val audioBytes = android.util.Base64.decode(mimoB64, android.util.Base64.DEFAULT)
                val audioDir = File(context.filesDir, "tts_audio")
                if (!audioDir.exists()) audioDir.mkdirs()
                val audioFile = File(audioDir, "${System.currentTimeMillis()}_mimo.mp3")
                java.io.FileOutputStream(audioFile).use { it.write(audioBytes) }
                return TtsResult(audioFile.absolutePath, null, true)
            }
            AppLogger.w(TAG, "MiMoTTS 响应中未找到音频数据")
            return TtsResult(null, null, false, "MiMoTTS 响应中未找到音频数据")
        }

        val json = JSONObject(bodyStr)

        val audioUrl = json.optString("url", "")
        if (audioUrl.isNotBlank()) return TtsResult(null, audioUrl, true)

```

```
92381            val audioB64 = json.optString("audio", "")
92382            if (audioB64.isNotBlank()) {
92383                val audioBytes = android.util.Base64.decode(audioB64, android.util.Base64.DEFAULT)
92384                val audioDir = File(context.filesDir, "tts_audio")
92385                if (!audioDir.exists()) audioDir.mkdirs()
92386                val audioFile = File(audioDir, "${System.currentTimeMillis()}.mp3")
92387                java.io.FileOutputStream(audioFile).use { it.write(audioBytes) }
92388                return TtsResult(audioFile.absolutePath, null, true)
92389            }
92390
92391            val dataArr = json.optJSONArray("data")
92392            if (dataArr != null && dataArr.length() > 0) {
92393                val firstData = dataArr.optJSONObject(0)
92394                val dataUrl = firstData?.optString("url", "") ?: ""
92395                val dataB64 = firstData?.optString("b64_json", "") ?: ""
92396                if (dataUrl.isNotBlank()) return TtsResult(null, dataUrl, true)
92397                if (dataB64.isNotBlank()) {
92398                    val audioBytes = android.util.Base64.decode(dataB64, android.util.Base64.DEFAULT)
92399                    val audioDir = File(context.filesDir, "tts_audio")
92400                    if (!audioDir.exists()) audioDir.mkdirs()
92401                    val audioFile = File(audioDir, "${System.currentTimeMillis()}.mp3")
92402                    java.io.FileOutputStream(audioFile).use { it.write(audioBytes) }
92403                    return TtsResult(audioFile.absolutePath, null, true)
92404                }
92405            }
92406        } catch (e: Exception) {
92407            AppLogger.e(TAG, "parseResponse JSON: ${e.message}")
92408        }
92409
92410        return TtsResult(null, null, false, "未找到音频内容")
92411    }
92412
92413    fun playAudio(path: String?, url: String?, onComplete: (() -> Unit)? = null) {
92414        stopPlayback()
92415        onPlaybackCompleteListener = onComplete
92416        try {
92417            mediaPlayer = MediaPlayer()
92418            when {
92419                !path.isNullOrBlank() -> {
92420                    currentlyPlayingPath = path
92421                    mediaPlayer?.setDataSource(path)
92422                }
92423                !url.isNullOrBlank() -> {
92424                    currentlyPlayingPath = url
92425                    mediaPlayer?.setDataSource(url)
92426                }
92427                else -> return
92428            }
92429            mediaPlayer?.prepareAsync()
92430            mediaPlayer?.setOnPreparedListener { mp ->
92431                mp.start()
92432            }
92433            mediaPlayer?.setOnCompletionListener { mp ->
92434                currentlyPlayingPath = null
92435                onPlaybackCompleteListener?.invoke()
92436                cleanupPlayer(mp)
92437            }
92438            mediaPlayer?.setOnErrorListener { mp, what, extra ->
92439                AppLogger.e(TAG, "MediaPlayer error: what=$what, extra=$extra")
92440                currentlyPlayingPath = null
92441                onPlaybackCompleteListener?.invoke()
```

```
92442                cleanupPlayer(mp)
92443                true
92444            }
92445        } catch (e: Exception) {
92446            AppLogger.e(TAG, "playAudio: ${e.message}")
92447            currentlyPlayingPath = null
92448            onPlaybackCompleteListener?.invoke()
92449        }
92450    }
92451
92452    fun stopPlayback() {
92453        val listener = onPlaybackCompleteListener
92454        onPlaybackCompleteListener = null
92455        try {
92456            mediaPlayer?.let {
92457                if (it.isPlaying) it.stop()
92458                it.reset()
92459                it.release()
92460            }
92461        } catch (_: Exception) {}
92462        mediaPlayer = null
92463        currentlyPlayingPath = null
92464        listener?.invoke()
92465    }
92466
92467    val isPlaying: Boolean
92468        get() = mediaPlayer?.isPlaying == true
92469
92470    val playingPath: String?
92471        get() = currentlyPlayingPath
92472
92473    private fun cleanupPlayer(mp: MediaPlayer?) {
92474        try {
92475            mp?.let {
92476                if (it.isPlaying) it.stop()
92477                it.reset()
92478                it.release()
92479            }
92480        } catch (_: Exception) {}
92481        if (mp == mediaPlayer) {
92482            mediaPlayer = null
92483            currentlyPlayingPath = null
92484        }
92485    }
92486
92487    fun cleanup() {
92488        stopPlayback()
92489        onPlaybackCompleteListener = null
92490    }
92491
92492    fun cleanupOldAudio(maxAgeMs: Long = 24 * 60 * 60 * 1000L) {
92493        try {
92494            val audioDir = File(context.filesDir, "tts_audio")
92495            if (!audioDir.exists()) return
92496            val cutoff = System.currentTimeMillis() - maxAgeMs
92497            audioDir.listFiles()?.forEach { file ->
92498                if (file.isFile && file.lastModified() < cutoff) {
92499                    file.delete()
92500                }
92501            }
92502        } catch (e: Exception) {
92503            AppLogger.e(TAG, "cleanupOldAudio: ${e.message}")
```

```
        }
    }
}
/** 语音管理器: TTS语音合成播放, 支持根据AI情绪调整语调和语速 */
package com.aicompanion.voice

import android.content.Context
import android.content.Intent
import android.media.MediaPlayer
import android.os.Bundle
import android.speech.RecognitionListener
import android.speech.RecognizerIntent
import android.speech.SpeechRecognizer
import android.speech.tts.TextToSpeech
import com.aicompanion.config.AppConfig
import com.aicompanion.models.Emotion
import com.aicompanion.settings.SettingsManager
import com.aicompanion.util.AppLogger
import java.util.*

class VoiceManager(private val context: Context) {

    private var speechRecognizer: SpeechRecognizer? = null
    private var textToSpeech: TextToSpeech? = null
    private var mediaPlayer: MediaPlayer? = null
    private var isListening = false
    @Volatile private var isTTSReady = false
    private data class PendingSpeechData(
        val text: String,
        val emotion: Emotion,
        val pitchOffset: Float,
        val rateOffset: Float,
        val timestamp: Long = System.currentTimeMillis(),  // 新增时间戳
        val id: String = java.util.UUID.randomUUID().toString().take(8)  // 新增唯一ID用于确认移除
    )

    private val pendingSpeechQueue = java.util.concurrent.ConcurrentLinkedDeque<PendingSpeechData>()

    companion object {
        private const val TAG = "VoiceManager"
    }

    init {
        try {
            if (SpeechRecognizer.isRecognitionAvailable(context)) {
                speechRecognizer = SpeechRecognizer.createSpeechRecognizer(context)
            }
        } catch (e: Exception) {
            AppLogger.e(TAG, "Failed to create speech recognizer: ${e.message}", e)
        }

        try {
            textToSpeech = TextToSpeech(context.applicationContext) { status ->
                if (status == TextToSpeech.SUCCESS) {
                    isTTSReady = true
                    try {
                        val cn = Locale.CHINESE
                        if (textToSpeech?.isLanguageAvailable(cn) == TextToSpeech.LANG_AVAILABLE ||
                            textToSpeech?.isLanguageAvailable(cn) == TextToSpeech.LANG_COUNTRY_AVAILABLE) {
                            textToSpeech?.language = cn
                        } else {
                            textToSpeech?.language = Locale.getDefault()
```

```
                }
            } catch (e: Exception) {
                AppLogger.w(TAG, "TTS language setting failed: ${e.message}")
            }
            AppLogger.w(TAG, "TTS初始化成功")
            var isFirst = true
            while (pendingSpeechQueue.isNotEmpty()) {
                val data = pendingSpeechQueue.removeFirst()
                speakInternal(data.text, data.emotion, data.pitchOffset, data.rateOffset, if (isFirst) TextToSpeech.QUEUE_FLUSH else TextToS
                isFirst = false
            }
        } else {
            AppLogger.w(TAG, "TTS初始化失败 status=$status")
            isTTSReady = false
        }
    }
} catch (e: Exception) {
    AppLogger.w(TAG, "Failed to create TTS: ${e.message}")
    isTTSReady = false
}
}

fun startListening(onResult: (String) -> Unit) {
    if (isListening) return
    val recognizer = speechRecognizer
    if (recognizer == null) {
        // AppLogger.w(TAG, "SpeechRecognizer not initialized, cannot start listening")
        return
    }

    isListening = true

    try {
        val intent = Intent(RecognizerIntent.ACTION_RECOGNIZE_SPEECH).apply {
            putExtra(RecognizerIntent.EXTRA_LANGUAGE_MODEL, RecognizerIntent.LANGUAGE_MODEL_FREE_FORM)
            putExtra(RecognizerIntent.EXTRA_LANGUAGE, Locale.CHINESE.toString())
            putExtra(RecognizerIntent.EXTRA_PARTIAL_RESULTS, true)
            putExtra(RecognizerIntent.EXTRA_MAX_RESULTS, 1)
        }

        recognizer.setRecognitionListener(object : RecognitionListener {
            override fun onReadyForSpeech(params: Bundle?) {}
            override fun onBeginningOfSpeech() {}
            override fun onRmsChanged(rmsdB: Float) {}
            override fun onBufferReceived(buffer: ByteArray?) {}
            override fun onEndOfSpeech() {}
            override fun onError(error: Int) {
                isListening = false
                val errorStr = when (error) {
                    SpeechRecognizer.ERROR_AUDIO -> "音频错误"
                    SpeechRecognizer.ERROR_CLIENT -> "客户端错误"
                    SpeechRecognizer.ERROR_INSUFFICIENT_PERMISSIONS -> "权限不足"
                    SpeechRecognizer.ERROR_NETWORK -> "网络错误"
                    SpeechRecognizer.ERROR_NETWORK_TIMEOUT -> "网络超时"
                    SpeechRecognizer.ERROR_NO_MATCH -> "未匹配到语音"
                    SpeechRecognizer.ERROR_RECOGNIZER_BUSY -> "识别器忙碌"
                    SpeechRecognizer.ERROR_SERVER -> "服务器错误"
                    SpeechRecognizer.ERROR_SPEECH_TIMEOUT -> "未检测到语音"
                    else -> "未知错误($error)"
                }
                AppLogger.w(TAG, "Speech recognition error: $errorStr")
            }
```

```
92628                override fun onResults(results: Bundle?) {
92629                    isListening = false
92630                    val matches = results?.getStringArrayList(SpeechRecognizer.RESULTS_RECOGNITION)
92631                    if (!matches.isNullOrEmpty()) {
92632                        onResult(matches[0])
92633                    }
92634                }
92635                override fun onPartialResults(partialResults: Bundle?) {}
92636                override fun onEvent(eventType: Int, params: Bundle?) {}
92637            })
92638
92639            recognizer.startListening(intent)
92640        } catch (e: Exception) {
92641            AppLogger.e(TAG, "Failed to start listening: ${e.message}", e)
92642            isListening = false
92643        }
92644    }
92645
92646    fun stopListening() {
92647        isListening = false
92648        try {
92649            speechRecognizer?.stopListening()
92650        } catch (e: Exception) {
92651            AppLogger.w(TAG, "Error stopping listening: ${e.message}", e)
92652        }
92653    }
92654
92655    private var utteranceCompleteListener: (() -> Unit)? = null
92656
92657    fun setOnUtteranceCompleteListener(listener: (() -> Unit)?) {
92658        utteranceCompleteListener = listener
92659    }
92660
92661    /**
92662     * 清理过期消息(超过配置时间)
92663     */
92664    fun cleanupExpiredQueue() {
92665        val now = System.currentTimeMillis()
92666        while (pendingSpeechQueue.isNotEmpty()) {
92667            val first = pendingSpeechQueue.peek()
92668            if (first == null) break
92669
92670            if (now - first.timestamp > AppConfig.VOICE_PENDING_QUEUE_EXPIRE_MS) {
92671                val removed = pendingSpeechQueue.poll()
92672                // 确保移除的是同一个对象（防止并发导致peek和poll操作不同对象）
92673                if (removed != null && removed.id == first.id) {
92674                    AppLogger.d(TAG, "Removed expired pending speech: ${removed.text.take(20)}...")
92675                }
92676            } else {
92677                break
92678            }
92679        }
92680    }
92681
92682    fun speak(text: String, emotion: Emotion = Emotion.NEUTRAL, emotionPitchOffset: Float = 0f, emotionRateOffset: Float = 0f) {
92683        cleanupExpiredQueue()  // 每次播放前清理过期消息
92684        if (text.isBlank()) return
92685        // 过滤括号内的动作/状态描述，避免TTS读出(微笑)(点头)等内容
92686        val cleanText = filterActionText(text)
92687        if (cleanText.isBlank()) return
92688        if (!isTTSReady) {
```

```
92689            if (pendingSpeechQueue.size < AppConfig.VOICE_PENDING_QUEUE_MAX) {
92690                AppLogger.w(TAG, "TTS未就绪，缓存语音等待初始化完成: ${cleanText.take(30)}...")
92691                pendingSpeechQueue.addLast(PendingSpeechData(cleanText, emotion, emotionPitchOffset, emotionRateOffset))
92692            } else {
92693                AppLogger.w(TAG, "TTS未就绪且缓存已满，丢弃: ${cleanText.take(30)}...")
92694            }
92695            return
92696        }
92697        speakInternal(cleanText, emotion, emotionPitchOffset, emotionRateOffset, TextToSpeech.QUEUE_FLUSH)
92698    }
92699
92700    /**
92701     * 过滤文本中的动作/状态描述（括号内容），返回纯净的可朗读文本
92702     * 支持格式：(微笑) （轻轻点头） *叹气* 【思考】 等
92703     */
92704    private fun filterActionText(text: String): String {
92705        var result = text
92706        // 移除中文全角括号 （...）
92707        result = result.replace("（[^）]*）".toRegex(), "")
92708        // 移除英文半角括号 (...)
92709        result = result.replace("\\([^)]*\\)".toRegex(), "")
92710        // 移除星号动作标记 *...*
92711        result = result.replace("\\*[^*]*\\*".toRegex(), "")
92712        // 移除方括号动作标记 [...]
92713        result = result.replace("\\[[^\\]]*\\]".toRegex(), "")
92714        // 清理多余空格（连续空格→单空格，首尾空格）
92715        result = result.replace("\\s+".toRegex(), " ").trim()
92716        return result
92717    }
92718
92719    fun stopSpeaking() {
92720        try {
92721            pendingSpeechQueue.clear()
92722            textToSpeech?.stop()
92723        } catch (e: Exception) {
92724            AppLogger.e(TAG, "stopSpeaking failed: ${e.message}", e)
92725        }
92726    }
92727
92728    fun isSpeaking(): Boolean {
92729        return try {
92730            textToSpeech?.isSpeaking == true
92731        } catch (e: Exception) {
92732            false
92733        }
92734    }
92735
92736    private fun speakInternal(text: String, emotion: Emotion, emotionPitchOffset: Float, emotionRateOffset: Float, queueMode: Int) {
92737        try {
92738            val sm = SettingsManager(context)
92739            val basePitch = sm.ttsPitch
92740            val baseRate = sm.ttsRate
92741
92742            val (emotionPitchDelta, emotionRateDelta) = when (emotion) {
92743                Emotion.HAPPY -> 0.2f to 0.1f
92744                Emotion.SAD -> -0.2f to -0.15f
92745                Emotion.ANGRY -> 0.1f to 0.15f
92746                Emotion.SURPRISED -> 0.3f to 0.1f
92747                Emotion.TSUNDERE -> 0.05f to 0f
92748                else -> 0f to 0f
92749            }
92750
```

```
92751            val pitch = (basePitch + emotionPitchDelta + emotionPitchOffset).coerceIn(0.5f, 2.0f)
92752            val rate = (baseRate + emotionRateDelta + emotionRateOffset).coerceIn(0.5f, 2.0f)
92753
92754            textToSpeech?.setPitch(pitch)
92755            textToSpeech?.setSpeechRate(rate)
92756            val utteranceId = UUID.randomUUID().toString()
92757            try {
92758                textToSpeech?.setOnUtteranceProgressListener(object : android.speech.tts.UtteranceProgressListener() {
92759                    override fun onStart(utteranceId: String?) {}
92760                    override fun onError(utteranceId: String?) {
92761                        utteranceCompleteListener?.invoke()
92762                    }
92763                    override fun onDone(utteranceId: String?) {
92764                        utteranceCompleteListener?.invoke()
92765                    }
92766                })
92767            } catch (_: Exception) {}
92768            textToSpeech?.speak(text, queueMode, null, utteranceId)
92769        } catch (e: Exception) {
92770            AppLogger.e(TAG, "TTS speak failed: ${e.message}", e)
92771        }
92772    }
92773
92774    fun playSpeech(audioUrl: String, emotion: Emotion = Emotion.NEUTRAL) {
92775        if (audioUrl.isBlank()) {
92776            AppLogger.w(TAG, "Empty audio URL")
92777            return
92778        }
92779        try {
92780            cleanupMediaPlayer()
92781            mediaPlayer = MediaPlayer().apply {
92782                setDataSource(audioUrl)
92783                prepareAsync()
92784                setOnPreparedListener { start() }
92785                setOnCompletionListener { cleanupMediaPlayer() }
92786                setOnErrorListener { _, what, extra ->
92787                    AppLogger.e(TAG, "MediaPlayer error: what=$what, extra=$extra")
92788                    cleanupMediaPlayer()
92789                    true
92790                }
92791            }
92792        } catch (e: Exception) {
92793            AppLogger.e(TAG, "Failed to play speech: ${e.message}", e)
92794            cleanupMediaPlayer()
92795        }
92796    }
92797
92798    private fun cleanupMediaPlayer() {
92799        try {
92800            if (mediaPlayer != null) {
92801                if (mediaPlayer?.isPlaying == true) {
92802                    mediaPlayer?.stop()
92803                }
92804                mediaPlayer?.reset()
92805                mediaPlayer?.release()
92806            }
92807        } catch (e: Exception) {
92808            AppLogger.w(TAG, "Error cleaning up MediaPlayer: ${e.message}", e)
92809        }
92810        mediaPlayer = null
92811    }
92812
```

```
    fun cleanup() {
        pendingSpeechQueue.clear()  // 清空队列
        isListening = false
        isTTSReady = false
        try {
            speechRecognizer?.destroy()
        } catch (e: Exception) {
            AppLogger.w(TAG, "Error destroying speech recognizer: ${e.message}", e)
        }
        speechRecognizer = null

        try {
            textToSpeech?.stop()
            textToSpeech?.shutdown()
        } catch (e: Exception) {
            AppLogger.w(TAG, "Error shutting down TTS: ${e.message}", e)
        }
        textToSpeech = null

        cleanupMediaPlayer()
    }
}
/** 开机自启广播: 监听Android开机广播, 自动启动后台服务 */
package com.aicompanion.wakeup

import android.content.BroadcastReceiver
import android.content.Context
import android.content.Intent
import android.os.Build
import android.util.Log
import com.aicompanion.services.BackgroundService
import com.aicompanion.settings.SettingsManager

class BootReceiver : BroadcastReceiver() {
    override fun onReceive(context: Context, intent: Intent) {
        if (intent.action == Intent.ACTION_BOOT_COMPLETED ||
            intent.action == "android.intent.action.QUICKBOOT_POWERON") {

            WakeUpScheduler.rescheduleOnBoot(context)

            val taskManager = com.aicompanion.wakeup.WakeUpTaskManager(context)
            taskManager.rescheduleOnBoot()

            val settingsManager = SettingsManager(context)
            if (settingsManager.autoStart && settingsManager.backgroundRunning) {
                try {
                    val serviceIntent = Intent(context, BackgroundService::class.java)
                    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
                        context.startForegroundService(serviceIntent)
                    } else {
                        context.startService(serviceIntent)
                    }
                    Log.d("BootReceiver", "BackgroundService started on boot")
                } catch (e: Exception) {
                    Log.e("BootReceiver", "Failed to start BackgroundService: ${e.message}", e)
                }
            }
        }
    }
}
package com.aicompanion.wakeup

```

```
import android.app.AlertDialog
import android.app.TimePickerDialog
import android.os.Bundle
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import android.widget.EditText
import android.widget.ImageView
import android.widget.Switch
import android.widget.TextView
import android.widget.Toast
import androidx.appcompat.app.AppCompatActivity
import androidx.recyclerview.widget.LinearLayoutManager
import androidx.recyclerview.widget.RecyclerView
import com.aicompanion.R
import com.google.android.material.floatingactionbutton.FloatingActionButton

class WakeUpActivity : AppCompatActivity() {

    private lateinit var taskManager: WakeUpTaskManager
    private lateinit var recyclerView: RecyclerView
    private lateinit var adapter: TaskAdapter

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_wakeup)

        taskManager = WakeUpTaskManager(this)
        taskManager.load()

        recyclerView = findViewById(R.id.rv_wakeup_tasks)
        adapter = TaskAdapter(taskManager.getAllTasks())
        recyclerView.layoutManager = LinearLayoutManager(this)
        recyclerView.adapter = adapter

        findViewById<FloatingActionButton>(R.id.fab_add_task).setOnClickListener {
            com.aicompanion.anim.AnimeUtils.pulse(it)
            showAddTaskDialog()
        }

        findViewById<View>(R.id.btn_back_wakeup).setOnClickListener {
            com.aicompanion.anim.AnimeUtils.pulse(it)
            finish()
        }

        applyTheme()
    }

    override fun onResume() {
        super.onResume()
        refreshList()
    }

    private fun refreshList() {
        adapter.updateData(taskManager.getAllTasks())
    }

    private fun applyTheme() {
        val scheme = com.aicompanion.theme.ThemeManager.getCurrentScheme(this)
        try {
            findViewById<View>(R.id.wakeup_root)?.setBackgroundColor(android.graphics.Color.parseColor(scheme.backgroundDark))
```

```
92936         } catch (_: Exception) {}
92937     }
92938
92939     private fun showAddTaskDialog() {
92940         val view = LayoutInflater.from(this).inflate(R.layout.dialog_add_wakeup_task, null)
92941         val etName = view.findViewById<EditText>(R.id.et_task_name)
92942         val etDesc = view.findViewById<EditText>(R.id.et_task_desc)
92943         var selectedHour = 9
92944         var selectedMinute = 0
92945
92946         val btnTime = view.findViewById<View>(R.id.btn_pick_time)
92947         val tvTime = view.findViewById<TextView>(R.id.tv_selected_time)
92948         btnTime.setOnClickListener {
92949             TimePickerDialog(this, { _, h, m ->
92950                 selectedHour = h
92951                 selectedMinute = m
92952                 tvTime.text = String.format("%02d:%02d", h, m)
92953             }, selectedHour, selectedMinute, true).show()
92954         }
92955
92956         AlertDialog.Builder(this)
92957             .setTitle("新建唤醒任务")
92958             .setView(view)
92959             .setPositiveButton("创建") { _, _ ->
92960                 val name = etName.text.toString().trim()
92961                 if (name.isBlank()) {
92962                     Toast.makeText(this, "请输入任务名", Toast.LENGTH_SHORT).show()
92963                     return@setPositiveButton
92964                 }
92965                 val task = WakeUpTask(
92966                     name = name,
92967                     description = etDesc.text.toString().trim(),
92968                     hour = selectedHour,
92969                     minute = selectedMinute,
92970                     enabled = true
92971                 )
92972                 taskManager.addTask(task)
92973                 refreshList()
92974                 Toast.makeText(this, "任务「$name」已创建", Toast.LENGTH_SHORT).show()
92975             }
92976             .setNegativeButton("取消", null)
92977             .show()
92978     }
92979
92980     private fun showEditTaskDialog(task: WakeUpTask) {
92981         val view = LayoutInflater.from(this).inflate(R.layout.dialog_add_wakeup_task, null)
92982         val etName = view.findViewById<EditText>(R.id.et_task_name)
92983         val etDesc = view.findViewById<EditText>(R.id.et_task_desc)
92984         var selectedHour = task.hour
92985         var selectedMinute = task.minute
92986
92987         etName.setText(task.name)
92988         etDesc.setText(task.description)
92989
92990         val btnTime = view.findViewById<View>(R.id.btn_pick_time)
92991         val tvTime = view.findViewById<TextView>(R.id.tv_selected_time)
92992         tvTime.text = String.format("%02d:%02d", task.hour, task.minute)
92993         btnTime.setOnClickListener {
92994             TimePickerDialog(this, { _, h, m ->
92995                 selectedHour = h
92996                 selectedMinute = m
92997                 tvTime.text = String.format("%02d:%02d", h, m)
```

```
92998            }, selectedHour, selectedMinute, true).show()
92999        }
93000
93001        val dialog = AlertDialog.Builder(this)
93002            .setTitle("编辑任务")
93003            .setView(view)
93004            .setPositiveButton("保存") { _, _ ->
93005                val name = etName.text.toString().trim()
93006                if (name.isBlank()) {
93007                    Toast.makeText(this, "请输入任务名", Toast.LENGTH_SHORT).show()
93008                    return@setPositiveButton
93009                }
93010                taskManager.updateTask(task.id) {
93011                    it.copy(name = name, description = etDesc.text.toString().trim(), hour = selectedHour, minute = selectedMinute)
93012                }
93013                refreshList()
93014            }
93015            .setNegativeButton("取消", null)
93016
93017        if (!task.isDefault) {
93018            dialog.setNeutralButton("删除") { _, _ ->
93019                taskManager.deleteTask(task.id)
93020                refreshList()
93021                Toast.makeText(this, "任务已删除", Toast.LENGTH_SHORT).show()
93022            }
93023        }
93024
93025        dialog.show()
93026    }
93027
93028    inner class TaskAdapter(private var items: List<WakeUpTask>) : RecyclerView.Adapter<TaskAdapter.VH>() {
93029        inner class VH(view: View) : RecyclerView.ViewHolder(view) {
93030            val tvName: TextView = view.findViewById(R.id.tv_task_name)
93031            val tvDesc: TextView = view.findViewById(R.id.tv_task_desc)
93032            val tvTime: TextView = view.findViewById(R.id.tv_task_time)
93033            val switchEnabled: Switch = view.findViewById(R.id.switch_task_enabled)
93034            val btnEdit: View = view.findViewById(R.id.btn_edit_task)
93035        }
93036
93037        override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): VH {
93038            val view = LayoutInflater.from(this@WakeUpActivity)
93039                .inflate(R.layout.item_wakeup_task, parent, false)
93040            return VH(view)
93041        }
93042
93043        override fun onBindViewHolder(holder: VH, position: Int) {
93044            val task = items[position]
93045            holder.tvName.text = task.name
93046            holder.tvDesc.text = task.description.ifBlank { "暂无描述" }
93047            holder.tvTime.text = String.format("%02d:%02d", task.hour, task.minute)
93048            holder.switchEnabled.setOnCheckedChangeListener(null)
93049            holder.switchEnabled.isChecked = task.enabled
93050
93051            holder.switchEnabled.setOnCheckedChangeListener { _, isChecked ->
93052                taskManager.toggleTask(task.id, isChecked)
93053            }
93054
93055            holder.btnEdit.setOnClickListener {
93056                showEditTaskDialog(task)
93057            }
93058
93059            holder.itemView.setOnLongClickListener {
```

```
            if (!task.isDefault) {
                AlertDialog.Builder(this@WakeUpActivity)
                    .setTitle("删除任务")
                    .setMessage("确定要删除「${task.name}」吗？")
                    .setPositiveButton("删除") { _, _ ->
                        taskManager.deleteTask(task.id)
                        refreshList()
                    }
                    .setNegativeButton("取消", null)
                    .show()
            }
            true
        }
    }

    override fun getItemCount() = items.size

    fun updateData(newItems: List<WakeUpTask>) {
        items = newItems
        notifyDataSetChanged()
    }
}
}
/** 唤醒调度器: 使用WorkManager定时唤醒后台服务, 确保AI桌宠保持活跃 */
package com.aicompanion.wakeup

import android.app.AlarmManager
import android.app.PendingIntent
import android.content.BroadcastReceiver
import android.content.Context
import android.content.Intent
import android.os.Build
import android.util.Log
import androidx.core.app.NotificationCompat
import androidx.core.app.NotificationManagerCompat
import com.aicompanion.R
import com.aicompanion.network.ApiClient
import com.aicompanion.ui.MainActivity
import com.aicompanion.models.ChatResponse
import org.json.JSONArray
import org.json.JSONObject
import java.util.*

class WakeUpReceiver : BroadcastReceiver() {
    override fun onReceive(context: Context, intent: Intent) {
        if (intent.action == ACTION_WAKE_UP) {
            val taskName = intent.getStringExtra("task_name") ?: ""
            val taskDesc = intent.getStringExtra("task_description") ?: ""
            if (taskName.isNotBlank()) {
                showTaskNotification(context, taskName, taskDesc)
            } else {
                showWakeUpNotification(context, true)
            }
            val taskManager = WakeUpTaskManager(context)
            taskManager.load()
            taskManager.scheduleAll()

            if (WakeUpScheduler.isWakeupEnabled(context)) {
                val (hour, minute) = WakeUpScheduler.getWakeupTime(context)
                WakeUpScheduler.scheduleWakeup(context, hour, minute)
            }
        }
```

```
93122    }
93123
93124    private fun showWakeUpNotification(context: Context, tryAI: Boolean = true) {
93125        val channelId = "ai_wakeup_channel"
93126        val notificationManager = NotificationManagerCompat.from(context)
93127
93128        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
93129            val channel = android.app.NotificationChannel(
93130                channelId,
93131                "AI唤醒",
93132                android.app.NotificationManager.IMPORTANCE_HIGH
93133            ).apply {
93134                description = "AI主动发起对话的提醒"
93135            }
93136            notificationManager.createNotificationChannel(channel)
93137        }
93138
93139        val pendingIntent = PendingIntent.getActivity(
93140            context,
93141            0,
93142            Intent(context, MainActivity::class.java).apply {
93143                flags = Intent.FLAG_ACTIVITY_NEW_TASK or Intent.FLAG_ACTIVITY_CLEAR_TOP
93144                putExtra("auto_wakeup", true)
93145            },
93146            PendingIntent.FLAG_UPDATE_CURRENT or PendingIntent.FLAG_IMMUTABLE
93147        )
93148
93149        val prefs = context.getSharedPreferences("companion_settings", Context.MODE_PRIVATE)
93150        val customMessage = prefs.getString("wake_message", null)
93151
93152        val activeId = context.getSharedPreferences("app_prefs", Context.MODE_PRIVATE)
93153            .getString("active_persona_id", "default") ?: "default"
93154        val personaPrefs = context.getSharedPreferences("persona_data_$activeId", Context.MODE_PRIVATE)
93155        val aiName = personaPrefs.getString("persona_name", null) ?: "星尘"
93156
93157        val baseGreeting = customMessage ?: run {
93158            val hour = Calendar.getInstance().get(Calendar.HOUR_OF_DAY)
93159            when {
93160                hour < 6 -> "夜深了，星尘还在等你..."
93161                hour < 12 -> "早上好！星尘想和你说说话~"
93162                hour < 18 -> "下午好呀！今天过得怎么样？"
93163                else -> "晚上好，今天辛苦啦~"
93164            }
93165        }
93166
93167        var notificationText = baseGreeting
93168
93169        if (tryAI) {
93170            val settingsPrefs = context.getSharedPreferences("companion_settings", Context.MODE_PRIVATE)
93171            val apiUrl = settingsPrefs.getString("chat_api_url", "") ?: ""
93172            val apiModel = settingsPrefs.getString("chat_model", "gpt-4o-mini") ?: "gpt-4o-mini"
93173            val llmTemp = settingsPrefs.getFloat("llm_temperature", 1.05f)
93174            val llmTopP = settingsPrefs.getFloat("llm_top_p", 0.92f)
93175            val llmFreqPenalty = settingsPrefs.getFloat("llm_frequency_penalty", 0.35f)
93176            val llmPresPenalty = settingsPrefs.getFloat("llm_presence_penalty", 0.5f)
93177            val llmMaxTokens = settingsPrefs.getInt("llm_max_tokens", 500)
93178            val apiProvider = settingsPrefs.getString("api_provider", "custom") ?: "custom"
93179            val apiKey = try {
93180                com.aicompanion.settings.SecurePrefsHelper.get(context)
93181                    .getString("chat_api_key", "") ?: ""
93182            } catch (e: Exception) {
```

```
93183            com.aicompanion.util.AppLogger.e("WakeUpReceiver", "读取API密钥失败: ${e.javaClass.simpleName}: ${e.message}")
93184            ""
93185        }
93186
93187        if (apiUrl.isNotBlank()) {
93188            Thread {
93189                try {
93190                    val aiClient = ApiClient(apiUrl, apiKey, apiModel,
93191                        llmTemp, llmTopP, llmFreqPenalty, llmPresPenalty, llmMaxTokens,
93192                        apiProvider)
93193                    val hour = Calendar.getInstance().get(Calendar.HOUR_OF_DAY)
93194                    val timeHint = when {
93195                        hour < 6 -> "凌晨"
93196                        hour < 12 -> "早上"
93197                        hour < 18 -> "下午"
93198                        else -> "晚上"
93199                    }
93200                    val activePersonaId = context.getSharedPreferences("app_prefs", Context.MODE_PRIVATE)
93201                        .getString("active_persona_id", "default") ?: "default"
93202                    val pm = com.aicompanion.persona.PersonaManager(context)
93203                    pm.load()
93204                    val persona = pm.getPersona(activePersonaId)
93205                    val personaPrompt = if (persona != null) {
93206                        buildString {
93207                            append("你是「${persona.name}」。")
93208                            if (persona.personality.isNotBlank()) append("\n性格：${persona.personality}")
93209                            if (persona.speechStyle.isNotBlank()) append("\n说话风格：${persona.speechStyle}")
93210                            if (persona.prompt.isNotBlank()) append("\n${persona.prompt}")
93211                        }
93212                    } else {
93213                        "你是「星尘」。"
93214                    }
93215                    val result = aiClient.sendProactiveChat(
93216                        persona?.name ?: "星尘",
93217                        personaPrompt,
93218                        "现在是$timeHint，你想主动找用户打个招呼。回复1-2句话，自然可爱，语气像认识很久的朋友。不需要自我介绍。直接说想说的
93219                        "（用户现在没在看手机，你想主动找ta说话）"
93220                    )
93221                    if (result != null && result.text.isNotBlank()) {
93222                        val aiText = result.text
93223                        if (aiText.length <= 60) {
93224                            notificationText = aiText
93225                            val notification = NotificationCompat.Builder(context, channelId)
93226                                .setSmallIcon(android.R.drawable.ic_dialog_info)
93227                                .setContentTitle("$aiName 唤醒")
93228                                .setContentText(notificationText)
93229                                .setStyle(NotificationCompat.BigTextStyle().bigText(notificationText))
93230                                .setPriority(NotificationCompat.PRIORITY_HIGH)
93231                                .setContentIntent(pendingIntent)
93232                                .setAutoCancel(true)
93233                                .build()
93234                            notificationManager.notify(NOTIFICATION_ID_WAKEUP, notification)
93235                            return@Thread
93236                        }
93237                    }
93238                } catch (e: Exception) {
93239                    Log.d("WakeUp", "AI wakeup failed: ${e.message}")
93240                }
93241            }.apply { start() }
93242        }
93243    }
93244
```

```
93245         val notification = NotificationCompat.Builder(context, channelId)
93246             .setSmallIcon(android.R.drawable.ic_dialog_info)
93247             .setContentTitle("$aiName 唤醒")
93248             .setContentText(notificationText)
93249             .setStyle(NotificationCompat.BigTextStyle().bigText(notificationText))
93250             .setPriority(NotificationCompat.PRIORITY_HIGH)
93251             .setContentIntent(pendingIntent)
93252             .setAutoCancel(true)
93253             .build()
93254 
93255         notificationManager.notify(NOTIFICATION_ID_WAKEUP, notification)
93256     }
93257 
93258     private fun showTaskNotification(context: Context, taskName: String, taskDesc: String) {
93259         val channelId = "ai_wakeup_channel"
93260         val notificationManager = NotificationManagerCompat.from(context)
93261 
93262         if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
93263             val channel = android.app.NotificationChannel(
93264                 channelId,
93265                 "AI唤醒",
93266                 android.app.NotificationManager.IMPORTANCE_HIGH
93267             ).apply {
93268                 description = "AI定时任务提醒"
93269                 enableVibration(true)
93270             }
93271             notificationManager.createNotificationChannel(channel)
93272         }
93273 
93274         val activeId2 = context.getSharedPreferences("app_prefs", Context.MODE_PRIVATE)
93275             .getString("active_persona_id", "default") ?: "default"
93276         val personaPrefs2 = context.getSharedPreferences("persona_data_$activeId2", Context.MODE_PRIVATE)
93277         val aiName = personaPrefs2.getString("persona_name", null)
93278             ?: context.getSharedPreferences("app_prefs", Context.MODE_PRIVATE).getString("ai_name", "星尘") ?: "星尘"
93279 
93280         val pendingIntent = PendingIntent.getActivity(
93281             context,
93282             0,
93283             Intent(context, MainActivity::class.java).apply {
93284                 flags = Intent.FLAG_ACTIVITY_NEW_TASK or Intent.FLAG_ACTIVITY_CLEAR_TOP
93285                 putExtra("auto_wakeup", true)
93286             },
93287             PendingIntent.FLAG_UPDATE_CURRENT or PendingIntent.FLAG_IMMUTABLE
93288         )
93289 
93290         val notification = NotificationCompat.Builder(context, channelId)
93291             .setSmallIcon(android.R.drawable.ic_dialog_info)
93292             .setContentTitle("$aiName：$taskName")
93293             .setContentText(taskDesc)
93294             .setStyle(NotificationCompat.BigTextStyle().bigText(taskDesc))
93295             .setPriority(NotificationCompat.PRIORITY_HIGH)
93296             .setCategory(NotificationCompat.CATEGORY_MESSAGE)
93297             .setContentIntent(pendingIntent)
93298             .setAutoCancel(true)
93299             .setTimeoutAfter(10000)
93300             .build()
93301 
93302         notificationManager.notify(taskName.hashCode() and 0x7FFFFFFF, notification)
93303     }
93304 
93305     companion object {
93306         const val ACTION_WAKE_UP = "com.aicompanion.ACTION_WAKE_UP"
```

```
93307         const val NOTIFICATION_ID_WAKEUP = 1001
93308     }
93309 }
93310
93311 object WakeUpScheduler {
93312     private const val PREFS_NAME = "companion_settings"
93313     private const val KEY_WAKEUP_ENABLED = "wake_enabled"
93314     private const val KEY_WAKEUP_HOUR = "wake_hour"
93315     private const val KEY_WAKEUP_MINUTE = "wake_minute"
93316
93317     fun isWakeupEnabled(context: Context): Boolean {
93318         return context.getSharedPreferences(PREFS_NAME, Context.MODE_PRIVATE)
93319             .getBoolean(KEY_WAKEUP_ENABLED, false)
93320     }
93321
93322     fun getWakeupTime(context: Context): Pair<Int, Int> {
93323         val prefs = context.getSharedPreferences(PREFS_NAME, Context.MODE_PRIVATE)
93324         val hour = prefs.getInt(KEY_WAKEUP_HOUR, 9)
93325         val minute = prefs.getInt(KEY_WAKEUP_MINUTE, 0)
93326         return Pair(hour, minute)
93327     }
93328
93329     fun setWakeupSettings(context: Context, enabled: Boolean, hour: Int, minute: Int) {
93330         context.getSharedPreferences(PREFS_NAME, Context.MODE_PRIVATE).edit().apply {
93331             putBoolean(KEY_WAKEUP_ENABLED, enabled)
93332             putInt(KEY_WAKEUP_HOUR, hour)
93333             putInt(KEY_WAKEUP_MINUTE, minute)
93334             apply()
93335         }
93336         if (enabled) {
93337             scheduleWakeup(context, hour, minute)
93338         } else {
93339             cancelWakeup(context)
93340         }
93341     }
93342
93343     fun scheduleWakeup(context: Context, hour: Int, minute: Int) {
93344         val alarmManager = context.getSystemService(Context.ALARM_SERVICE) as AlarmManager
93345
93346         val calendar = Calendar.getInstance().apply {
93347             set(Calendar.HOUR_OF_DAY, hour)
93348             set(Calendar.MINUTE, minute)
93349             set(Calendar.SECOND, 0)
93350             set(Calendar.MILLISECOND, 0)
93351             if (timeInMillis <= System.currentTimeMillis()) {
93352                 add(Calendar.DAY_OF_MONTH, 1)
93353             }
93354         }
93355
93356         val intent = Intent(context, WakeUpReceiver::class.java).apply {
93357             action = WakeUpReceiver.ACTION_WAKE_UP
93358         }
93359
93360         val pendingIntent = PendingIntent.getBroadcast(
93361             context,
93362             0,
93363             intent,
93364             PendingIntent.FLAG_UPDATE_CURRENT or PendingIntent.FLAG_IMMUTABLE
93365         )
93366
93367         if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.M) {
93368             alarmManager.setExactAndAllowWhileIdle(
```

```
                AlarmManager.RTC_WAKEUP,
                calendar.timeInMillis,
                pendingIntent
            )
        } else {
            alarmManager.setExact(AlarmManager.RTC_WAKEUP, calendar.timeInMillis, pendingIntent)
        }
    }

    fun cancelWakeup(context: Context) {
        val alarmManager = context.getSystemService(Context.ALARM_SERVICE) as AlarmManager

        val intent = Intent(context, WakeUpReceiver::class.java).apply {
            action = WakeUpReceiver.ACTION_WAKE_UP
        }

        val pendingIntent = PendingIntent.getBroadcast(
            context,
            0,
            intent,
            PendingIntent.FLAG_UPDATE_CURRENT or PendingIntent.FLAG_IMMUTABLE
        )

        alarmManager.cancel(pendingIntent)
    }

    fun rescheduleOnBoot(context: Context) {
        if (isWakeupEnabled(context)) {
            val (hour, minute) = getWakeupTime(context)
            scheduleWakeup(context, hour, minute)
        }
    }
}
package com.aicompanion.wakeup

import android.app.AlarmManager
import android.app.PendingIntent
import android.content.Context
import android.content.Intent
import android.os.Build
import com.aicompanion.util.AppLogger
import org.json.JSONArray
import org.json.JSONObject
import java.io.File
import java.util.UUID

data class WakeUpTask(
    val id: String = UUID.randomUUID().toString(),
    val name: String,
    val description: String,
    val hour: Int,
    val minute: Int,
    val enabled: Boolean = true,
    val isDefault: Boolean = false
) {
    fun toJson(): JSONObject = JSONObject().apply {
        put("id", id)
        put("name", name)
        put("description", description)
        put("hour", hour)
        put("minute", minute)
        put("enabled", enabled)
```

```
93431         put("isDefault", isDefault)
93432     }
93433
93434     companion object {
93435         fun fromJson(obj: JSONObject): WakeUpTask = WakeUpTask(
93436             id = obj.optString("id", UUID.randomUUID().toString()),
93437             name = obj.optString("name", "唤醒任务"),
93438             description = obj.optString("description", ""),
93439             hour = obj.optInt("hour", 9),
93440             minute = obj.optInt("minute", 0),
93441             enabled = obj.optBoolean("enabled", true),
93442             isDefault = obj.optBoolean("isDefault", false)
93443         )
93444     }
93445 }
93446
93447 class WakeUpTaskManager(private val context: Context) {
93448     companion object {
93449         private const val TAG = "WakeUpTaskManager"
93450         private const val FILE_NAME = "wakeup_tasks.json"
93451         private const val KEY_INITIALIZED = "wakeup_tasks_initialized"
93452         private const val PREFS_NAME = "wakeup_task_prefs"
93453     }
93454
93455     private val file = File(context.filesDir, FILE_NAME)
93456     private val tasks = mutableListOf<WakeUpTask>()
93457
93458     fun load() {
93459         tasks.clear()
93460         if (!file.exists()) {
93461             initDefaults()
93462             return
93463         }
93464         try {
93465             val json = JSONObject(file.readText())
93466             val arr = json.optJSONArray("tasks") ?: return
93467             for (i in 0 until arr.length()) {
93468                 tasks.add(WakeUpTask.fromJson(arr.getJSONObject(i)))
93469             }
93470         } catch (e: Exception) {
93471             AppLogger.e(TAG, "load failed: ${e.message}")
93472             initDefaults()
93473         }
93474     }
93475
93476     private fun initDefaults() {
93477         val prefs = context.getSharedPreferences(PREFS_NAME, Context.MODE_PRIVATE)
93478         if (prefs.getBoolean(KEY_INITIALIZED, false)) return
93479         tasks.add(WakeUpTask(
93480             id = "default_health_sleep",
93481             name = "健康监督",
93482             description = "该睡觉啦！早睡早起身体好~明天也要元气满满哦！",
93483             hour = 23,
93484             minute = 30,
93485             enabled = true,
93486             isDefault = true
93487         ))
93488         save()
93489         prefs.edit().putBoolean(KEY_INITIALIZED, true).apply()
93490     }
93491
```

```
    fun save() {
        try {
            val json = JSONObject().apply {
                put("tasks", JSONArray().apply {
                    tasks.forEach { put(it.toJson()) }
                })
            }
            file.writeText(json.toString())
        } catch (e: Exception) {
            AppLogger.e(TAG, "save failed: ${e.message}")
        }
    }

    fun getAllTasks(): List<WakeUpTask> = tasks.toList()

    fun addTask(task: WakeUpTask): WakeUpTask {
        tasks.add(task)
        save()
        scheduleAll()
        return task
    }

    fun updateTask(id: String, updater: (WakeUpTask) -> WakeUpTask): WakeUpTask? {
        val idx = tasks.indexOfFirst { it.id == id }
        if (idx < 0) return null
        val updated = updater(tasks[idx])
        tasks[idx] = updated
        save()
        scheduleAll()
        return updated
    }

    private var previousTaskIds: Set<String> = emptySet()

    fun deleteTask(id: String) {
        previousTaskIds = tasks.map { it.id }.toSet()
        tasks.removeAll { it.id == id }
        save()
        scheduleAll()
    }

    fun toggleTask(id: String, enabled: Boolean) {
        val idx = tasks.indexOfFirst { it.id == id }
        if (idx >= 0) {
            previousTaskIds = tasks.map { it.id }.toSet()
            tasks[idx] = tasks[idx].copy(enabled = enabled)
            save()
            scheduleAll()
        }
    }

    fun scheduleAll() {
        val alarmManager = context.getSystemService(Context.ALARM_SERVICE) as AlarmManager
        tasks.filter { it.enabled }.forEach { task ->
            scheduleTask(alarmManager, task)
        }
        cancelOrphanedAlarms(alarmManager)
    }

    private fun scheduleTask(alarmManager: AlarmManager, task: WakeUpTask) {
        val calendar = java.util.Calendar.getInstance().apply {
            set(java.util.Calendar.HOUR_OF_DAY, task.hour)
```

```
            set(java.util.Calendar.MINUTE, task.minute)
            set(java.util.Calendar.SECOND, 0)
            set(java.util.Calendar.MILLISECOND, 0)
            if (timeInMillis <= System.currentTimeMillis()) {
                add(java.util.Calendar.DAY_OF_MONTH, 1)
            }
        }

        val intent = Intent(context, WakeUpReceiver::class.java).apply {
            action = WakeUpReceiver.ACTION_WAKE_UP
            putExtra("task_id", task.id)
            putExtra("task_name", task.name)
            putExtra("task_description", task.description)
        }

        val requestCode = task.id.hashCode() and 0x7FFFFFFF
        val pendingIntent = PendingIntent.getBroadcast(
            context,
            requestCode,
            intent,
            PendingIntent.FLAG_UPDATE_CURRENT or PendingIntent.FLAG_IMMUTABLE
        )

        try {
            if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.M) {
                alarmManager.setExactAndAllowWhileIdle(
                    AlarmManager.RTC_WAKEUP,
                    calendar.timeInMillis,
                    pendingIntent
                )
            } else {
                alarmManager.setExact(AlarmManager.RTC_WAKEUP, calendar.timeInMillis, pendingIntent)
            }
        } catch (e: Exception) {
            AppLogger.e(TAG, "scheduleTask failed: ${e.message}")
        }
    }

    private fun cancelOrphanedAlarms(alarmManager: AlarmManager) {
        val validIds = tasks.filter { it.enabled }.map { it.id }.toSet()
        val oldIds = if (previousTaskIds.isNotEmpty()) previousTaskIds else loadTaskIdsFromPrefs()
        for (taskId in oldIds) {
            if (taskId !in validIds) {
                val requestCode = taskId.hashCode() and 0x7FFFFFFF
                val intent = Intent(context, WakeUpReceiver::class.java).apply {
                    action = WakeUpReceiver.ACTION_WAKE_UP
                    putExtra("task_id", taskId)
                }
                val pendingIntent = PendingIntent.getBroadcast(
                    context, requestCode, intent,
                    PendingIntent.FLAG_NO_CREATE or PendingIntent.FLAG_IMMUTABLE
                )
                if (pendingIntent != null) {
                    alarmManager.cancel(pendingIntent)
                    pendingIntent.cancel()
                }
            }
        }
        previousTaskIds = emptySet()
    }

    private fun loadTaskIdsFromPrefs(): Set<String> {
```

```
93616         return try {
93617             if (!file.exists()) return emptySet()
93618             val json = JSONObject(file.readText())
93619             val arr = json.optJSONArray("tasks") ?: return emptySet()
93620             val ids = mutableSetOf<String>()
93621             for (i in 0 until arr.length()) {
93622                 ids.add(arr.getJSONObject(i).optString("id", ""))
93623             }
93624             ids.filter { it.isNotBlank() }.toSet()
93625         } catch (_: Exception) { emptySet() }
93626     }
93627
93628     fun rescheduleOnBoot() {
93629         load()
93630         scheduleAll()
93631     }
93632 }
```